

3DGEER: 3D GAUSSIAN RENDERING MADE EXACT AND EFFICIENT FOR GENERIC CAMERAS

Anonymous authors
Anonymous submission

ABSTRACT

3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) strikes a compelling balance between rendering quality and efficiency, yet this efficiency rests on approximating each 3D Gaussian by its 2D image-plane projection—an approximation whose error grows sharply under generic, large field-of-view (FoV) cameras and systematically degrades reconstruction accuracy. Recent extensions relax individual facets of this limitation, but none has simultaneously attained both *projective exactness* and *real-time efficiency* for general camera models. We introduce 3DGEER, a geometrically exact and efficient Gaussian rendering framework that resolves this long-standing tension. Proceeding from first principles, we derive a closed-form expression for the transmittance integral of Gaussian density along a ray, which yields exact forward rendering and differentiable optimization under arbitrary camera models. To preserve efficiency, we introduce the *Particle Bounding Frustum (PBF)*, which establishes tight ray–Gaussian association directly in 3D without BVH traversal, and the *Bipolar Equiangular Projection (BEAP)*, a unified FoV representation that accelerates association and, through more uniform ray sampling, improves reconstruction quality. Across both pinhole and fisheye datasets, 3DGEER outperforms prior methods on every reported metric, runs $5\times$ faster than existing projective-exact ray-based baselines, and generalizes to wider FoVs unseen during training—establishing a new state of the art in real-time radiance field rendering.

1 INTRODUCTION

Initiated by 3D Gaussian Splatting (3DGS) (Kerbl et al., 2023), volumetric particle rendering methods formulated under the splatting paradigm (Yu et al., 2024a; Yan et al., 2024; Lu et al., 2024; Kheradmand et al., 2024; Charatan et al., 2024; Mallick et al., 2024) have become a dominant approach to real-time neural rendering, owing to their combination of high visual fidelity and fast rendering. This efficiency originates in EWA splatting (Zwicker et al., 2001), which applies a local affine approximation to project each 3D Gaussian as a 2D Gaussian on the image plane. The approximation is accurate in narrow field-of-view (FoV) regimes, but it breaks down under wide FoVs: nonlinear distortion drives the true projection away from any symmetric 2D Gaussian, and the resulting error is most severe in the fisheye imagery pervasive in robotics and autonomous driving (Rashed et al., 2021; Courbon et al., 2007; Kumar et al., 2023; Zhang et al., 2016; Yogamani et al., 2019; 2024; Sekkat et al., 2022). Recent works enable differentiable rendering under specific fisheye camera models (Liao et al., 2024) or broader camera families (Huang et al., 2024b), yet each remains anchored to a 2D projective-space approximation and therefore stops short of an exact closed-form formulation.

A distinct line of reasoning questions whether exact rendering is necessary at all: Celarek et al. (2025) argues that scaling the number of Gaussians (up to 1M) suffices to absorb the projective approximation error of splatting. This evidence, however, is confined to small object-centric scenes. In large-scale reconstruction the required Gaussian budget varies drastically across objects and frustums, rendering such brute-force scaling prohibitively costly; and, as we demonstrate empirically, the projective error under generic distorted cameras is substantially larger, so that increasing the Gaussian count alone cannot close the quality gap (see Fig. 5).

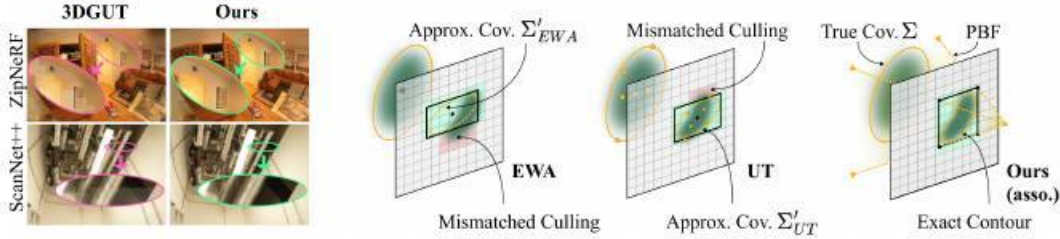


Figure 1: **Linear Approximation Error in Ray-Particle Association.** (Left) Grid-line artifacts induced by inaccurate UT-based association. (Right) Diagram contrasting our association with prior schemes: rather than passing through an intermediate conic approximation, our method computes the exact bounding structure directly from the true 3D covariance.

A parallel line of work confronts volumetric particle rendering directly through a ray-tracing/marching formulation (Yu et al. [2024b], Condor et al. [2025], Mai et al. [2025], Moënnelocoz et al. [2024], Talegaonkar et al. [2025], Gu et al. [2025]). Such formulations natively accommodate large-FoV camera models and are, by construction, free of projective approximation error. Their central obstacle is efficiency, and specifically the cost of associating each rendering ray with the 3D particles that contribute to it. Considerable effort has therefore gone into accelerating this association—most notably the advanced Bounding Volume Hierarchies (BVH) adopted in EVER (Mai et al. [2025]) and 3DGRT (Moënnelocoz et al. [2024])—yet the inherent algorithmic complexity of BVH traversal and its resistance to parallelization have kept pure ray-based methods well short of 3DGS-level throughput. The hybrid 3DGUT (Wu et al. [2025]) instead pairs ray-based rendering with rasterization to accelerate association, but the projective approximation it inherits at the association stage reintroduces exactly the geometric error that ray tracing was meant to avoid, compromising projective exactness and inducing grid-line artifacts (see Fig. 1 Left). No prior method, then, attains both the geometric exactness of ray-based rendering and the efficiency of splatting simultaneously—the precise gap this work closes.

We introduce 3DGEER, a 3D Gaussian Exact and Efficient Rendering method that attains *exactness in projective geometry* (formally defined in Sec. A) while retaining *efficiency comparable to splatting*, and does so uniformly across generic cameras—both perspective and a range of fisheye projection models. The framework rests on three tightly coupled contributions that jointly realize this goal: (i) *An exact, numerically stable rendering formulation.* We revisit the heuristic maximum-response function of Gaussians (Keselman & Hebert [2022]) from first principles and establish its precise connection to projective exactness; from this connection we derive a differentiable rendering framework whose forward and backward passes are numerically stable by construction. (ii) *Exact yet efficient ray-particle association.* We recast association as a mapping between Camera Sub-Frustums (CSFs) and Particle Bounding Frustums (PBFs)—the frustum-level analogue of the tile-AABB mapping in 3DGS—and derive a closed-form solution for the PBF that tightly bounds each 3D Gaussian, delivering association that is both geometrically exact and highly efficient. (iii) *Uniform angular supervision via BEAP.* We propose to sample supervision rays uniformly in a novel Bipolar Equiangular Projection (BEAP) space, which aligns the image-space partitioning with the underlying CSFs and, empirically, improves reconstruction quality over conventional pinhole and equidistant projections.

Extensive experiments spanning both fisheye and pinhole datasets—ScanNet++ (Yeshwanth et al. [2023]), Zip-NeRF (Barron et al. [2023]), Aria (Lv et al. [2025]), and MipNeRF (Barron et al. [2021])—establish that 3DGEER consistently outperforms prior and concurrent methods, with PSNR gains of 0.9–4.8 dB under challenging wide-FoV conditions and 0.3–2.0 dB on pinhole views. Notably, when trained with narrow-FoV data alone, 3DGEER still surpasses splatting-based methods on wide-FoV cameras, evidencing a strong capacity to generalize to extremely wide-FoV rendering.

2 RELATED WORKS

2.1 SPLATTING-BASED GAUSSIAN RENDERING

Most 3DGS variants (Yu et al. [2024a], Zhang et al. [2024], Rota Bulò et al. [2024], Gao et al. [2025], Zhao et al. [2025], Kerbl et al. [2024], Hou et al. [2025], Liu et al. [2025]) inherit the original EWA

Table 1: **Summary of Particle Rendering Methods w.r.t. Projective Exactness.** The upper block lists splatting-based methods; the lower block lists ray-based approaches, several of which nonetheless retain a projective-approximate association stage (e.g., EWA or the Unscented Transform). 3DGEER is the only method that jointly attains large-FoV support, high frame rate, and projective exactness in both rendering and association.

Method	Render	Particle Asso. Method	Particle Scope	Particle Type	Large FoV	High FPS	Exact Meth.
3DGS [Kerbl et al., 2023]	Splat.	EWA	Img.	3D Gauss.	✗	✓	✗
FisheyeGS [Liao et al., 2024]	Splat.	EWA	Img.	3D Gauss.	✓	✓	✗
GS++ [Huang et al., 2024b]	Splat.	EWA	Img.	3D Gauss.	✗	✓	✗
2DGS [Huang et al., 2024a]	Ray	Splat Bound.	Img.	2D Gauss.	✗	✓	✓
GOF [Yu et al., 2024b]	Ray	EWA	Img.	3D Gauss.	✗	✗	✗
EVER [Mai et al., 2025]	Ray	BVH	Scene	3D Ellip.	✓	✗	✓
3DGRT [Moënné-Loccoz et al., 2024]	Ray	BVH	Scene	3D Gauss.	✓	✗	✓
3DGUT [Wu et al., 2025]	Ray	UT	Img.	3D Gauss.	✓	✓	✗
HTGS [Hahlbohm et al., 2025]	Ray	Splat Bound.	Img.	3D Gauss.	✗	✓	✓
3DGEER (Ours)	Ray	PBF	Frust.	3D Gauss.	✓	✓	✓

splatting strategy [Zwicker et al., 2001, Ren et al., 2002], projecting each 3D Gaussian onto the image plane and modeling its footprint as a 2D conic. To extend differentiable rendering to large-FoV, distorted camera models, methods such as FisheyeGS [Liao et al., 2024] and GS++ [Huang et al., 2024b] estimate the Jacobian of equidistant or spherical projections and correct the 2D mean and covariance of the projected Gaussians accordingly. Because these corrections remain first-order Taylor expansions of a strongly nonlinear projection, however, the neglected higher-order terms become non-negligible under wide FoVs, producing substantial approximation error and degraded reconstruction quality.

The root cause is geometric: under any nonlinear camera model—distorted or even ideal pin-hole—the true projection of a 3D Gaussian is not a symmetric 2D Gaussian. Any framework that approximates an inherently nonlinear projection with linear projective geometry therefore incurs an approximation error that cannot be eliminated within that framework. 3DGEER sidesteps this error entirely, aggregating Gaussian density directly along rays in a volumetric formulation and thereby dispensing with projective approximation altogether.

2.2 RAY-BASED GAUSSIAN RENDERING

Keselman & Hebert [2022] posits that a ray interacts with a Gaussian density field at the point of maximum intensity response. Building on this heuristic, 3DGRT [Moënné-Loccoz et al., 2024] constructs a 3D Gaussian ray-tracing framework that samples directly in ray space without explicit projection, thereby avoiding wide-FoV distortion. Its ray-particle association, however, depends on costly scene-level BVH partitioning, which imposes substantial overhead and drives frame rates well below real time.

EVER [Mai et al., 2025] instead replaces Gaussians with constant-density ellipsoids, which admit a closed-form transmittance-weighted ray integral; this substitution, however, both narrows the representational capacity relative to full 3D Gaussians and, through its continued reliance on BVH structures, remains limited in runtime. 2DGS [Huang et al., 2024a] substitutes 3D Gaussians with 2D surfels to obtain exact ray-surface intersections. HTGS [Hahlbohm et al., 2025] restores the full Gaussian expression while retaining 2DGS’s ray-splat association; because that association is confined to screen space, it cannot guarantee projective exactness under wider FoVs. In contrast, we derive closed-form bounds and a complete rendering pipeline directly in a camera-agnostic angular domain (Sec. D), and we make the computational relationship to the 2DGS and HTGS parameterizations explicit in Sec. D.3.

3DGUT [Wu et al., 2025], a follow-up to 3DGRT, recovers 3DGS-level frame rates by adopting splatting-based particle association—but its acceleration hinges on the Unscented Transform (UT), a sampling-based approximation of the 2D conic in projective image space. GOF [Yu et al., 2024b] exhibits the same limitation, applying the EWA construction to fit 2D bounding conics and projective centers for association. Under extreme FoVs, the resulting imprecise AABB estimates and mismatched tile-to-AABB associations manifest as visible grid-line artifacts (see Fig. 1 Left).

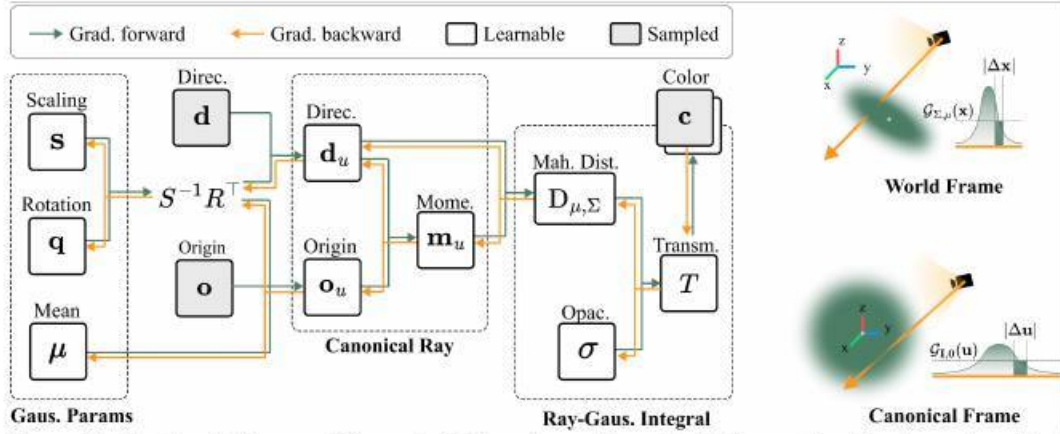


Figure 2: **Gradient Flow and Canonical Transformation.** (Left) Forward and backward gradient propagation linking the Gaussian parameters $\mathbf{s}$, $\mathbf{q}$, $\boldsymbol{\mu}$ and the opacity coefficient σ to the transmittance. (Right) Integration of 3D Gaussian density along a ray; the green box highlights the measure-preserving product of density and ray-segment length (see Eq. B.5).

3DGEER departs from all of these by solving ray-particle association directly at the frustum level in 3D space (see Fig. 1 Right), which preserves the geometric exactness of ray-based rendering across arbitrary camera models while sustaining rendering speeds on par with 3DGS. Tab. 1 consolidates representative methods that target projective approximation error and makes the resulting differences explicit across the dimensions that matter for exact, real-time, wide-FoV rendering.

3 METHOD

We present 3DGEER, a Gaussian rendering framework that delivers geometric exactness and real-time efficiency simultaneously under arbitrary FoV cameras. The design directly targets the two bottlenecks that have kept prior work from reaching both goals at once—the *projection approximation* intrinsic to splatting-based rendering and the *BVH overhead* intrinsic to ray-based rendering—and resolves them through three mutually reinforcing components:

- **Ray-based rendering formulation** (Sec. 3.1): a closed-form, differentiable solution for integrating Gaussian density along 3D rays that eliminates the linearization error of projection-based splatting at its source.
- **Particle Bounding Frustum (PBF)** (Sec. 3.2): a ray-particle association scheme that supplants BVH traversal with exact frustum-particle tests, achieving accuracy and efficiency together.
- **Bipolar Equiangular Projection (BEAP)** (Sec. 3.3): an image representation that unifies arbitrary FoV camera models, accelerates PBF computation, and improves reconstruction quality through more uniform ray sampling.

The Appendix supplies the supporting formal apparatus: (i) the preliminaries and notation (Sec. B.1); (ii) the complete proof underlying Sec. 3.1 which establishes the equivalence between the maximum-response heuristic and projective exactness (Sec. B); (iii) the full backward-gradient derivations together with their numerical-stability analysis (Sec. C); and (iv) the complete derivation of geometrically exact association (Sec. D).

3.1 RENDERING WITH EXACT PROJECTIVE GEOMETRY

The central obstacle to geometrically exact Gaussian rendering is obtaining a closed-form expression for the integral of density and color along a ray. Our strategy resolves this obstacle by mapping each anisotropic Gaussian in world space into a canonical coordinate system in which every Gaussian is isotropic; under this map the transmittance integral reduces to a measure-preserving change of variables and admits a compact closed-form solution.

Canonical transformation. For each Gaussian with mean μ , rotation R , and scaling S , we define the mapping

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (1)$$

which sends a canonical coordinate $\mathbf{u}$ to world coordinates $\mathbf{x}$. It follows that $\Delta\mathbf{x} = RS\Delta\mathbf{u}$, so the Jacobian determinant is absorbed into the density measure (see Fig. 2 Right), and every Gaussian collapses to the shared isotropic form

$$\mathcal{G}_{\mathbf{I},0}(\mathbf{u}) = \frac{1}{\rho} \exp(-\frac{1}{2}\|\mathbf{u}\|^2). \quad (2)$$

Closed-form transmittance. For a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, its canonical form is $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$ with

$$\mathbf{o}_u = S^{-1}R^\top(\mathbf{o} - \mu), \quad \mathbf{d}_u = S^{-1}R^\top\mathbf{d}. \quad (3)$$

The accumulated transmittance is then

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I},0}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp(-\frac{1}{2}D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d})), \quad (4)$$

where

$$D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d}) = \frac{\|\mathbf{o}_u \times \mathbf{d}_u\|^2}{\|\mathbf{d}_u\|^2} \quad (5)$$

is the squared perpendicular (Mahalanobis) distance from the ray to the Gaussian center.

Discussion. The transmittance thus depends solely on the ray’s distance to the Gaussian in canonical space, giving a closed-form solution that is at once efficient and geometrically exact. This expression coincides with the “maximum-response” heuristic invoked by prior work (Keselman & Hebert 2022; Moënné-Loccoz et al. 2024); our derivation, however, exposes the projective exactness that underlies it (Sec. B), promoting an empirical shortcut to a principled, first-principles result. Because we follow this closed-form derivation and its exact backward gradient chain (Fig. 2 Left; Sec. C), we further avoid by construction the numerical instabilities encountered by prior ray-based work (Yu et al. 2024b) and remove the need for redundant post-processing such as filtering out degenerate Gaussians.

3.2 RAY-PARTICLE ASSOCIATION WITH EXACT PROJECTIVE GEOMETRY

In any ray-based Gaussian renderer, both accuracy—especially under large-FoV camera models—and runtime efficiency hinge on the ray-particle association strategy, a component prior work has largely left underexamined. To identify which Gaussians contribute to each ray, we reduce association to a mapping between Camera Sub-Frustums (CSFs) and Particle Bounding Frustums (PBFs), where the PBF is the tight frustum enclosing each Gaussian at a prescribed distance from its center (Fig. 3). A Gaussian is assigned to a CSF precisely when its PBF intersects that CSF, and every Gaussian associated with a CSF is treated as a contributor to all rays within it.

To compute a PBF efficiently, we begin by defining two spherical angles (see Fig. E.1) in camera space:

$$\theta = \arctan\left(\frac{\mathbf{d}_{c,x}}{\mathbf{d}_{c,z}}\right), \quad \phi = \arctan\left(\frac{\mathbf{d}_{c,y}}{\mathbf{d}_{c,z}}\right), \quad (6)$$

where $(\mathbf{d}_{c,x}, \mathbf{d}_{c,y}, \mathbf{d}_{c,z})$ is the normalized direction vector toward the camera- (view-)space point $\mathbf{d}_c$. The angles θ and ϕ are the ray’s incidence angles in the horizontal (xz) and vertical (yz) planes,

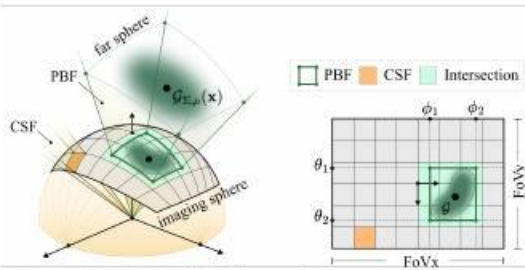


Figure 3: **PBF-CSF Association.** (Left) A PBF is defined by four planes tangent to the 3D Gaussian ellipsoid. (Right) The PBF-CSF intersection, unfolded onto the BEAP imaging plane.

respectively, each confined to $[-\frac{\pi}{2}, \frac{\pi}{2}]$. We then specify a PBF through four planes—two aligned with the θ bounds and two with the ϕ bounds (see Fig. 3):

$$\mathbf{g}_{\theta_{1,2}} = (-1, 0, \tan \theta_{1,2}, 0)^\top, \quad \mathbf{g}_{\phi_{1,2}} = (0, -1, \tan \phi_{1,2}, 0)^\top, \quad (7)$$

with $\theta_1 < \theta < \theta_2$ and $\phi_1 < \phi < \phi_2$ delimiting the angular extent.

Given a camera with extrinsic matrix $V = [R_c | \mathbf{t}_c] \in \text{SE}(3)$ and a canonical-to-world transformation matrix H (Eq. B.4), we transport these view-space angular limits into canonical space by applying the inverse transpose of the camera-to-canonical matrix $(VH)^{-1}$ to the plane parameters, which yields the constraints defining the PBF. Taking the θ bounds as the representative case, the canonical plane reads:

$$\mathbf{g}_{u_{1,2}} = (VH)^\top \mathbf{g}_{\theta_{1,2}} = -(VH)_0 + \tan \theta_{1,2} (VH)_2, \quad (8)$$

where $(VH)_i$ denotes the i -th row of the matrix, written as a column vector.

For a Gaussian ellipsoid defined by its λ -standard-deviation contour, we obtain the angular bounds of the local frustum by enforcing tangency between the transformed planes and the ellipsoidal contour, which reduces to the quadratic equations:

$$((\lambda^2, \lambda^2, \lambda^2, -1) \circ \mathbf{g}_u^\top) \cdot \mathbf{g}_u = 0, \quad (9)$$

where $\circ$ is the Hadamard (element-wise) product and λ governs the spread of the contour (we set $\lambda = 3$ throughout). The same constraint applied to $\mathbf{g}_v^\top$ yields the corresponding bounds $\phi_{1,2}$. The roots of these quadratics give the tangent values of the angular bounds $[\theta_1, \theta_2]$ and $[\phi_1, \phi_2]$ that delimit the PBF. Crucially, solving these quadratics under the canonical-space constraint operates on the Gaussian’s true camera-space 3D covariance Σ_c and 3D mean μ_c rather than on any projected surrogate. Concretely, for the θ bounds, $c = \tan \theta_{1,2}$ satisfies:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad \text{where} \quad \mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (10)$$

This formulation is inexpensive to solve (Sec. D.1) and, by operating on the true 3D covariance, dispenses with both the intermediate conic approximation of EWA and UT and the screen-space constraints that bound the admissible FoV in HTGS and 2DGS.

3.3 BIPOLAR EQUIANGULAR PROJECTION (BEAP)

The BEAP image representation is motivated by three complementary properties. First, it represents large-FoV imagery without incurring any FoV loss. Second, it endows image tiles and their corresponding CSFs with an identical parameterization in terms of two θ and two ϕ angles, which makes PBF association highly efficient and maps naturally onto GPU parallelism. Third, the induced ray sampling is far more uniform in 3D than that of conventional projective rendering, and this uniformity empirically improves both training efficiency and reconstruction quality. Together, these properties make BEAP the representation that binds the exact rendering and PBF association components into a single wide-FoV pipeline.

Concretely, we sample rays uniformly in the spherical angular coordinates (θ, ϕ) (Eq. 6) and map them back to a discrete $w \times h$ image grid through a linear transformation—a scaling followed by a center shift:

$$(x, y) = \left\lfloor \frac{w\theta}{\text{FoV}_x} + \frac{w+1}{2}, \frac{h\phi}{\text{FoV}_y} + \frac{h+1}{2} \right\rfloor, \quad (11)$$

where $\text{FoV}_x, \text{FoV}_y$ are the camera’s horizontal and vertical fields of view. As Fig. 4 shows, uniform sampling in (θ, ϕ) yields markedly more balanced spatial coverage of the view frustum than the two standard alternatives: projective sampling over-allocates rays to the periphery, whereas equidistant projection over-concentrates them near the image center.

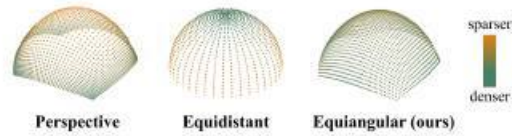


Figure 4: **Ray Distributions under Different Projections.** Pixels from three imaging spaces are projected onto the unit sphere to compare their ray distributions; among the three, BEAP attains the most uniform angular coverage.

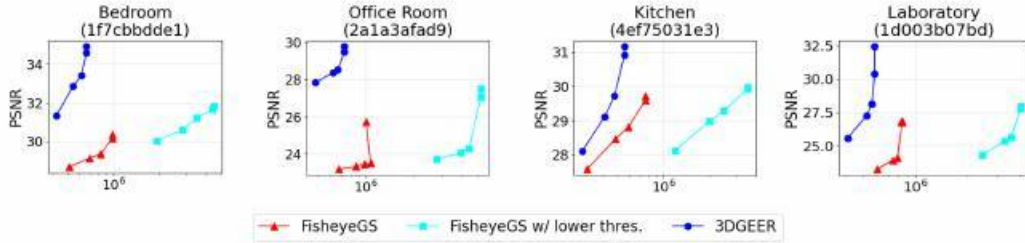


Figure 5: **PSNR $\uparrow$ vs. Gaussian Count.** Increasing the Gaussian budget does not close the projective-exactness gap: brute-force scaling of the splatting baseline plateaus well below 3DGEER (full metrics in Fig. K.3 and Tab. K.3).



Figure 6: **Extreme-FoV Generalization.** With fewer Gaussians, 3DGEER reconstructs complete scenes, whereas FisheyeGS degrades severely at these unseen wide FoVs.

4 EXPERIMENTS

We evaluate 3DGEER across multiple datasets spanning both pinhole and fisheye camera models, so that its behavior is assessed over the full range of FoVs it is designed to handle. Unless otherwise stated, all experiments use an RTX 4090 GPU with 64 GB of RAM as the default configuration; full implementation details appear in Sec. G.

Datasets. We assess performance on generic distorted cameras—the regime in which projective error is most consequential—using all scenes of Fisheye ZipNeRF (Barron et al., 2023), five scenes of ScanNet++ (Yeshwanth et al., 2023), and two of Aria (Lv et al., 2025). The ZipNeRF and ScanNet++ cameras have a 180° diagonal FoV, and Aria a 110° circular FoV. For the pinhole regime we use Pinhole ZipNeRF and MipNeRF360 (Barron et al., 2021). Together these datasets exercise the method across narrow-, wide-, and cross-camera conditions.

Evaluation Metrics and Protocol. We report PSNR $\uparrow$, SSIM $\uparrow$, LPIPS $\downarrow$, runtime (FPS $\uparrow$), and Gaussian count $\downarrow$. To eliminate representation bias, every method’s outputs are reprojected to the native image space and evaluated over the full FoV after 30k training iterations, so that all methods are compared on identical, camera-native ground.

4.1 SCALING GAUSSIANS VS. PROJECTIVE EXACTNESS IN WIDE-FOV CAMERAS

Celarek et al. (2025) contends that merely scaling the number of Gaussians (up to 1M) is enough to absorb the linear approximation gap between splatting- and ray-based methods. That claim, however, rests on small object-centric scenes and leaves the challenge of wide-FoV distortion untouched. To test it directly, we scale the splatting-based fisheye method FisheyeGS on ScanNet++ by lowering its densification threshold, driving the model to 3.6–5M Gaussians (on an H200).

The result, in Fig. 5 is decisive. At matched Gaussian counts, the FisheyeGS curve (red) and the 3DGEER curve (blue) remain separated by a clear PSNR gap across the full densification schedule (4k–30k iterations, beyond which densification saturates)—a gap attributable to projective exactness

Table 2: **Comparison on the ScanNet++ Dataset.** Each method is trained on either the full FoV or the central region alone, then evaluated on the full, central, and peripheral regions to expose robustness under varying distortion. Region-wise metrics are pixel-mask weighted; region-wise LPIPS is omitted as it is not separable by region (Tab. K.2 reports the full statistics). 3DGEER attains the best full-FoV PSNR, SSIM, and LPIPS, and—trained on central crops alone—still surpasses FisheyeGS across all regions.

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	PSNR ↑	SSIM ↑
3DGS	Central	Persp.	-	-	-	31.26	0.948	-	-
FisheyeGS	Full	Eq.	27.81	0.946	0.139	32.44	0.956	23.28	0.914
EVER	Full	KB	29.47	0.924	0.167	29.93	0.924	28.72	0.925
3DGUT	Full	Eq.	30.64	0.944	0.150	31.87	0.945	28.84	0.937
3DGEER (Ours)	Central	BEAP	29.93	0.949	0.130	32.86	0.954	26.64	0.930
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

Table 3: **Comparison on the ZipNeRF Dataset.** A cross-camera generalization protocol: each method is trained on either Fisheye (FE, 1/8 resolution) or Pinhole (PH, 1/4 resolution) data and evaluated separately on both Fisheye and Pinhole test sets (Tab. K.9 reports the full statistics). 3DGEER leads on PSNR, SSIM, and LPIPS in nearly every train/test combination, and its advantage is largest in the demanding pinhole-train/fisheye-test setting.

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
	FE	PH	FE	PH	FE	PH	FE	PH	FE	PH	FE	PH
Test Data	PSNR ↑		SSIM ↑		LPIPS ↓		PSNR ↑		SSIM ↑		LPIPS ↓	
Method	PSNR ↑		SSIM ↑		LPIPS ↓		PSNR ↑		SSIM ↑		LPIPS ↓	
FisheyeGS	23.18	26.44	0.858	0.868	0.211	0.239	19.43	26.61	0.791	0.889	0.247	0.212
EVER	25.17	26.14	0.880	0.851	0.153	0.237	22.90	26.45	0.835	0.862	0.207	0.222
3DGUT	24.77	25.59	0.879	0.804	0.183	0.324	18.61	25.96	0.662	0.841	0.300	0.270
3DGEER (Ours)	26.24	27.62	0.897	0.888	0.140	0.214	23.39	27.61	0.846	0.863	0.209	0.254

rather than model capacity. The scaled-up FisheyeGS (light blue) confirms this: despite a large increase in memory and computation, its PSNR plateaus at 29.3, whereas 3DGEER reaches 32.1 with only 550–700K Gaussians. Scaling capacity therefore cannot substitute for exactness; full statistics appear in Tab. K.3

4.2 COMPREHENSIVE EVALUATION ON ALL-FOV NOVEL VIEW SYNTHESIS

We first benchmark 3DGEER against representative baselines on both fisheye and pinhole datasets, augmenting the comparison with analysis tailored to each dataset.

ScanNet++. 3DGEER outperforms every baseline on all reported metrics (Tab. 2). Decomposing the FoV into central and peripheral regions (protocol in Sec. G) isolates the source of this margin: the competing methods render the center acceptably but degrade toward the periphery, exactly where projective error concentrates, whereas 3DGEER sustains high fidelity across the entire FoV, with sharper boundaries and fewer artifacts (Fig. K.4). The advantage is robust to a substantial reduction in training FoV: trained on central crops alone, 3DGEER still surpasses FisheyeGS in every region.

This robustness extends to extreme-FoV generalization (Fig. 6). Training at the native 180° diagonal FoV and testing at a much wider 180° circular FoV, 3DGEER reconstructs the complete scene with fewer Gaussians, while splatting-based methods such as FisheyeGS break down at the periphery (further examples under extreme FoV in Figs. K.1–K.2).

ZipNeRF. Because this dataset supplies frame-wise aligned pinhole and fisheye views, we go beyond standard fisheye evaluation and probe *cross-camera generalization*—training on fisheye and testing on pinhole, and vice versa. 3DGEER leads across metrics in Tab. 3 exploiting the large-FoV training data that geometrically exact ren-

Table 4: **Quantitative Results on the MipNeRF360 Dataset.** Alongside quality metrics, FPS is measured on an RTX 4090 as the default benchmark; figures taken from the original works are marked † (RTX 6000 Ada) and ‡ (RTX 5090). 3DGEER attains the best PSNR and LPIPS while running at splatting-comparable frame rates—over 5× faster than the projective-exact ray-based baselines EVER and 3DGRT.

Method	PSNR ↑	SSIM ↑	LPIPS ↓	FPS ↑
3DGS	27.21	0.815	0.214	343

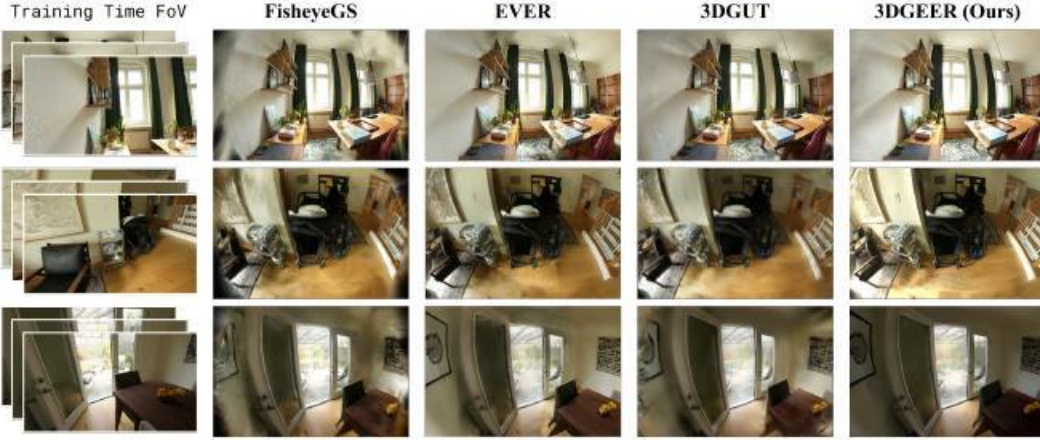


Figure 7: **Qualitative Results on ZipNeRF under Cross-Camera Generalization.** The gap is most pronounced in the challenging Pinhole (PH)-train / Fisheye (FE)-test setting, where 3DGEER holds a clear advantage.

Table 5: **Training-Time Transmittance Replacement on ScanNet++.** Substituting a splatting-based transmittance inflates the Gaussian count—densification compensating for the larger approximation error—yet a perceptual gap to our projective-exact formulation persists, confirming that the error cannot be densified away (Tab. K.7 reports the full statistics).

Transmittance Method	Full FoV			Central		Peripheral		Avg. Gaussian Number (k) ↓
	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	PSNR ↑	SSIM ↑	
3DGS Splats	22.86	0.878	0.177	27.16	0.884	18.56	0.855	1396.4
FisheyeGS Splats	27.90	0.948	0.141	32.49	0.957	23.36	0.917	920.6
Projective-Exact (Ours)	31.50	0.953	0.126	32.64	0.955	28.94	0.945	591.8

dering makes usable; even trained on pinhole data, it still delivers clear PSNR gains on both test views. The baselines fare far worse under domain shift: FisheyeGS is competitive only in the pinhole-to-pinhole case, and 3DGUT collapses on fisheye after pinhole training. Only EVER and 3DGEER generalize gracefully across cameras—both being geometrically exact—and 3DGEER dominates EVER in every setting. Qualitatively (Fig. 7), 3DGEER recovers finer detail and resists peripheral artifacts most visibly in the hardest pinhole-to-fisheye regime.

Finally, we verify that the gains under wide FoVs do not come at the expense of the standard narrow-FoV regime, evaluating 3DGEER on **MipNeRF360** (Tab. 4). 3DGEER sets a new state of the art across the reported metrics while occupying a point on the quality–efficiency frontier that no prior method reaches: it runs over $5\times$ faster than the exact ray-marching/tracing methods EVER and 3DGRT—making it the only projective-exact method with runtime comparable to 3DGS—and simultaneously surpasses the projective-approximation baselines 3DGS and 3DGUT on PSNR, SSIM, and LPIPS. Additional quantitative and qualitative results on **Aria**, an egocentric fisheye dataset (moderate 110° circular FoV) captured under low light, appear in Sec. J.

4.3 COMPREHENSIVE RUNTIME ANALYSIS

We next dissect runtime on the RTX 4090, comparing 3DGEER against the splatting-based methods 3DGS (on **MipNeRF360**) and FisheyeGS (on **ScanNet++**). While ray-based rendering is marginally more expensive in the rendering stage itself, PBF’s efficient ray–particle association sharply curtails

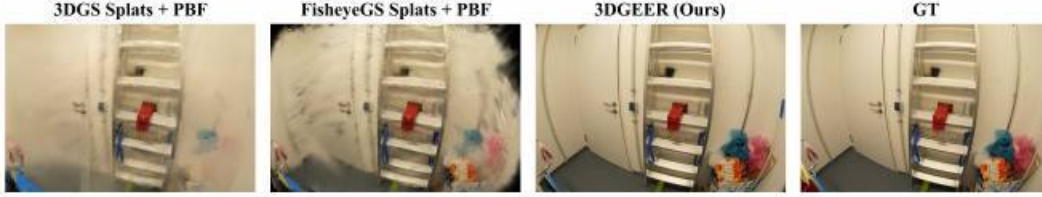


Figure 8: **Rendering-Time Transmittance Replacement.** Swapping the projective-exact transmittance for the splatting approximation at test time alone induces mismatched culling, isolating the rendering formulation as the cause.

Table 6: **BEAP vs. Alternative Supervision Schemes.** With the projective-exact integral and PBF held fixed, BEAP supervision outperforms perspective and equidistant sampling on every metric on ScanNet++ (Tab. K.8 reports the full statistics).

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGEER w/o BEAP	Central	Persp.	29.84	0.943	0.131	32.21	0.951	26.23	0.915
3DGEER w/o BEAP	Full	Persp.	21.11	0.853	0.300	21.32	0.855	20.46	0.846
3DGEER w/o BEAP	Full	Eq.	31.05	0.948	0.135	32.21	0.952	28.56	0.935
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

Gaussian duplication and sorting overhead (stage-wise timings in Tab. K.4), so the end-to-end efficiency matches that of highly optimized splatting pipelines.

The mechanism is the tightness of the PBF bounds: each tile is associated with only ~ 475 Gaussians on average, $3\text{--}5\times$ fewer than EWA or UT (Tab. K.6), which translates directly into the $2.5\text{--}5\times$ speedup measured at the association stage (Tab. K.5). Projection-based schemes (EWA, UT), by contrast, conservatively overestimate the screen-space extent, incurring excessive overdraw and inefficient memory access; even with SnugBox acceleration (Hanson et al. 2025) they remain limited by miscentered density approximations that often aggravate artifacts (Fig. 1). The benefit of PBF is thus not confined to speed—by operating on the true 3D covariance it also preserves geometric accuracy and numerical stability and removes projective approximation error outright. Tab. K.6 makes this concrete: even when rendering Gaussian fields trained under other association schemes, PBF remains artifact-free.

4.4 LOCAL AFFINE APPROXIMATION VS. PROJECTIVE-EXACT INTEGRAL

To isolate the contribution of the projective-exact transmittance formulation (Eq. 4), we replace it with the widely used local affine splat (the 2D conic of 3DGS and FisheyeGS) and examine the effect at both test and training time.

Rendering-time approximation. Substituting the splatting approximation for the transmittance at test time only—while retaining every projective-exact component during training—cleanly exposes the difference between the two rendering formulations. In this configuration the transmittance rests on the splat approximation while association remains exact, and the resulting inconsistency produces mismatched culling (Fig. 8). The experiment magnifies, and thereby explains, the gridline artifact reported for 3DGUT (Fig. 1), which arises from the mirror-image mismatch: ray-based transmittance paired with conic-approximate association.

Training-time approximation. When the splatting-based transmittance is used during training as well, with association held projective-exact (Tab. 5 ScanNet++), the optimizer and densification partly absorb both the approximation error and the cross-view inconsistency that splatting introduces. The elevated loss drives stronger gradients throughout training and inflates the Gaussian budget by $1.6\text{--}2.4\times$. Yet the perceptual gap to the exact formulation survives this compensation, demonstrating that densification alone cannot correct the approximation error.

4.5 BEAP ON/OFF STUDY

We isolate the effect of the supervision distribution by holding the projective-exact integral and PBF association fixed and varying only the ray-sampling scheme used for supervision (pinhole, equidistant, and BEAP) on **ScanNet++**. BEAP supervision is the best of the three on every metric (Tab. 6, Fig. K.5): its more uniform sampling supplies smoother, more consistent gradients and thus better convergence on fine detail.

The effect sharpens under pinhole cameras, where—at fixed resolution—how information is distributed across the image can matter more than the raw FoV. Training in the perspective domain, the full FoV can even trail the central region (Tab. 6 rows 1–2): large-FoV perspective images (Fig. K.2, *Left*) sample angle highly non-uniformly, concentrating informative content at the center while the less informative periphery dominates the image area and is disproportionately oversampled. BEAP’s uniform angular sampling removes precisely this imbalance.

5 CONCLUSIONS

We presented 3DGEER, a complete closed-form solution for projective-exact volumetric Gaussian rendering derived from first principles. Around this formulation we built an efficient PBF-based ray–particle association that sustains high-speed rendering without sacrificing projective exactness, and a BEAP image representation that delivers uniform, effective color supervision under wide-FoV cameras. The three components are mutually reinforcing: the exact ray integral supplies the geometry, PBF supplies the efficiency, and BEAP supplies the supervision that lets both operate across the full FoV. Extensive experiments across pinhole and fisheye datasets show that 3DGEER consistently outperforms state-of-the-art methods, with its largest margins in the challenging wide-FoV regimes where projective approximation error dominates—resolving the long-standing tension between exactness and efficiency in real-time radiance field rendering.

ETHICS STATEMENT

This work builds on publicly available datasets (ScanNet++, ZipNeRF, Aria, and MipNeRF) and does not involve any human subjects, private data, or personally identifiable information. The methods developed are intended for advancing 3D scene reconstruction and rendering research, with potential applications in XR, robotics and autonomous driving. We do not foresee direct harmful consequences, but we note that any misuse of high-fidelity 3D reconstruction techniques could raise privacy concerns. To mitigate this, we restrict our experiments to research datasets that are widely used in the vision community.

REPRODUCIBILITY STATEMENT

We make significant efforts to ensure reproducibility. All mathematical derivations and proofs are provided in the Appendix (Secs. [B](#), [C](#) and [D](#)). The datasets used in our experiments are publicly available (ScanNet++, ZipNeRF, Aria, and MipNeRF). Details of preprocessing, training protocols, and evaluation metrics are described in Sec. [G](#). Our implementation will be released with training and evaluation scripts upon publication to further facilitate reproducibility.

REFERENCES

- Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. In *IEEE International Conference on Computer Vision (ICCV)*, 2021.
- Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Zip-nerf: Anti-aliased grid-based neural radiance fields. In *IEEE International Conference on Computer Vision, (ICCV)*, 2023.
- Adam Celarek, George Kopanas, George Drettakis, Michael Wimmer, and Bernhard Kerbl. Does 3d gaussian splatting need accurate volumetric rendering? In *Computer Graphics Forum (CGF)*, 2025.
- David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, pp. 19457–19467, 2024.
- Jorge Condor, Sébastien Speierer, Lukas Bode, Aljaz Bozic, Simon Green, Piotr Didyk, and Adrián Jarabo. Don’t splat your gaussians: Volumetric ray-traced primitives for modeling and rendering scattering and emissive media. *ACM Transactions on Graphics (TOG)*, 2025.
- Jonathan Courbon, Youcef Mezouar, Laurent Eckt, and Philippe Martinet. A generic fisheye camera model for robotic applications. In *IEEE/RSJ International Conference on Intelligent Robots and Systems, (IROS)*, pp. 1683–1688, 2007.
- Daniel Duckworth, Peter Hedman, Christian Reiser, Peter Zhizhin, Jean-François Thibert, Mario Lucic, Richard Szeliski, and Jonathan T. Barron. SMERF: streamable memory efficient radiance fields for real-time large-scene exploration. *ACM Transactions on Graphics (TOG)*, 2024.
- Zhongpai Gao, Benjamin Planche, Meng Zheng, Anwesa Choudhuri, Terrence Chen, and Ziyang Wu. 6dgs: Enhanced direction-aware gaussian splatting for volumetric rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Chun Gu, Xiaofei Wei, Zixuan Zeng, Yuxuan Yao, and Li Zhang. Irgs: Inter-reflective gaussian splatting with 2d gaussian ray tracing. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10943–10952, 2025.
- Yuliang Guo, Sparsh Garg, S. Mahdi H. Miangoleh, Xinyu Huang, and Liu Ren. Depth any camera: Zero-shot metric depth estimation from any camera. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

-
- Florian Hahlbohm, Fabian Friederichs, Tim Weyrich, Linus Franke, Moritz Kappel, Susana Castillo, Marc Stamminger, Martin Eisemann, and Marcus Magnor. Efficient perspective-correct 3d gaussian splatting using hybrid transparency. In *Computer Graphics Forum (CGF)*, 2025.
- Alex Hanson, Allen Tu, Geng Lin, Vasu Singla, Matthias Zwicker, and Tom Goldstein. Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 21537–21546, 2025.
- Qiqi Hou, Randall Rauwendaal, Zifeng Li, Hoang Le, Farzad Farhadzadeh, Fatih Porikli, Alexei Bourd, and Amir Said. Sort-free gaussian splatting via weighted sum rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2d gaussian splatting for geometrically accurate radiance fields. In Andres Burbano, Denis Zorin, and Wojciech Jarosz (eds.), *ACM SIGGRAPH Conference Papers*, 2024a.
- Letian Huang, Jiayang Bai, Jie Guo, and Yanwen Guo. Gs++: Error analyzing and optimal gaussian splatting. *CoRR*, 2024b.
- Juho Kannala and Sami S Brandt. A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2006.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)*, 2023.
- Bernhard Kerbl, Andreas Meuleman, Georgios Kopanas, Michael Wimmer, Alexandre Lanvin, and George Drettakis. A hierarchical 3d gaussian representation for real-time rendering of very large datasets. *ACM Transactions on Graphics (TOG)*, 43(4):1–15, 2024.
- Leonid Keselman and Martial Hebert. Approximate differentiable rendering with algebraic surfaces. In *European Conference on Computer Vision (ECCV)*, 2022.
- Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Yang-Che Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:80965–80986, 2024.
- Varun Ravi Kumar, Ciarán Eising, Christian Witt, and Senthil Kumar Yogamani. Surround-view fisheye camera perception for automated driving: Overview, survey & challenges. *IEEE Transactions on Intelligent Transportation Systems*, 24(4):3638–3659, 2023.
- Zimu Liao, Siyan Chen, Rong Fu, Yi Wang, Zhongling Su, Hao Luo, Li Ma, Linning Xu, Bo Dai, Hengjie Li, et al. Fisheye-gs: Lightweight and extensible gaussian splatting module for fisheye cameras. *ECCV Workshop*, 2024.
- Changkun Liu, Shuai Chen, Yash Sanjay Bhalgat, Siyan HU, Ming Cheng, Zirui Wang, Victor Adrian Prisacariu, and Tristan Braud. Gs-cpr: Efficient camera pose refinement via 3d gaussian splatting. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-gs: Structured 3d gaussians for view-adaptive rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20654–20664, 2024.
- Zhaoyang Lv, Maurizio Monge, Ka Chen, Yufeng Zhu, Michael Goesele, Jakob Engel, Zhao Dong, and Richard Newcombe. Photoreal scene reconstruction from an egocentric device. In *ACM SIGGRAPH Conference Papers*, 2025.
- Alexander Mai, Peter Hedman, George Kopanas, Dor Verbin, David Futschik, Qiangeng Xu, Falko Kuester, Jonathan T Barron, and Yinda Zhang. EVER: exact volumetric ellipsoid rendering for real-time view synthesis. *IEEE International Conference on Computer Vision (ICCV)*, 2025.

- Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3dgs: High-quality radiance fields with limited resources. In *SIGGRAPH Asia Conference Papers*, 2024.
- Xiaoyang Mao, Lichan Hong, and Arie Kaufman. Splatting of curvilinear volumes. In *Proceedings Visualization '95*, pp. 61–68. IEEE, 1995.
- Christopher Mei and Patrick Rives. Single view point omnidirectional camera calibration from planar grids. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2007.
- Nicolas Moënné-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 3d gaussian ray tracing: Fast tracing of particle scenes. *ACM Transactions on Graphics (TOG)*, 2024.
- Hazem Rashed, Eslam Mohamed, Ganesh Sistu, Varun Ravi Kumar, Ciaran Eising, Ahmad El-Sallab, and Senthil Yogamani. Generalized object detection on fisheye cameras for autonomous driving: Dataset, representations and baseline. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, (WACV), pp. 2272–2280, 2021.
- Liu Ren, Hanspeter Pfister, and Matthias Zwicker. Object space ewa surface splatting: A hardware accelerated approach to high quality point rendering. In *Computer graphics forum (CGF)*, 2002.
- Samuel Rota Bulò, Lorenzo Porzi, and Peter Kotschieder. Revising densification in gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 347–362. Springer, 2024.
- Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- Ahmed Rida Sekkat, Yohan Dupuis, Varun Ravi Kumar, Hazem Rashed, Senthil Yogamani, Pascal Vasseur, and Paul Honeine. Synwoodscape: Synthetic surround-view fisheye camera dataset for autonomous driving. *IEEE Robotics and Automation Letters (RAL)*, 7(3):8502–8509, 2022.
- Chinmay Talegaonkar, Yash Belhe, Ravi Ramamoorthi, and Nicholas Antipa. Volumetrically consistent 3d gaussian rasterization. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10953–10963, 2025.
- Lee Westover. Footprint evaluation for volume rendering. In *Proceedings of the 17th annual conference on Computer graphics and interactive techniques*, pp. 367–376, 1990.
- Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moënné-Loccoz, and Zan Gojcic. 3dgt: Enabling distorted cameras and secondary rays in gaussian splatting. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-scale 3d gaussian splatting for anti-aliased rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 20923–20931, 2024.
- Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Senthil Yogamani, Ciarán Hughes, Jonathan Horgan, Ganesh Sistu, Padraig Varley, Derek O’Dea, Michal Uricár, Stefan Milz, Martin Simon, Karl Amende, et al. Woodscape: A multi-task, multi-camera fisheye dataset for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9308–9318, 2019.
- Senthil Yogamani, David Unger, Venkatraman Narayanan, and Varun Ravi Kumar. Fisheyebevseg: Surround view fisheye cameras based bird’s-eye view segmentation for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 1331–1334, 2024.
- Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3d gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, (CVPR), pp. 19447–19456, 2024a.

-
- Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics (TOG)*, 2024b.
- Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-gs: Density control with pixel-aware gradient for 3d gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 326–342. Springer, 2024.
- Zichao Zhang, Henri Rebecq, Christian Forster, and Davide Scaramuzza. Benefit of large field-of-view cameras for visual odometry. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 801–808. IEEE, 2016.
- Hexu Zhao, Haoyang Weng, Daohan Lu, Ang Li, Jinyang Li, Aurojit Panda, and Saining Xie. On scaling up 3d gaussian splatting training. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Ewa volume splatting. In *Proceedings Visualization, 2001. VIS'01.*, pp. 29–538. IEEE, 2001.

CONTENTS

1 Introduction	1
2 Related Works	2
2.1 Splatting-Based Gaussian Rendering	2
2.2 Ray-Based Gaussian Rendering	3
3 Method	4
3.1 Rendering with Exact Projective Geometry	4
3.2 Ray-Particle Association with Exact Projective Geometry	5
3.3 Bipolar Equiangular Projection (BEAP)	6
4 Experiments	7
4.1 Scaling Gaussians vs. Projective Exactness in Wide-FoV Cameras	7
4.2 Comprehensive Evaluation on All-FoV Novel View Synthesis	8
4.3 Comprehensive Runtime Analysis	9
4.4 Local Affine Approximation vs. Projective-Exact Integral	10
4.5 BEAP On/Off Study	11
5 Conclusions	11
a Ethics Statement	12
b Reproducibility Statement	12
c LLM Usage Statement	18
A Appendix: Exactness and Limitation	18
B Appendix: Ray-Gaussian Transmittance with Projective Exactness	18
B.1 Preliminaries And Notations	18
B.2 Differentiable Rendering with Projective Exactness	19
B.3 Ray Integral of Isotropic 3D Gaussian	20
C Appendix: Cookbook of Gradient Computation	21
C.1 Chain Rule	21
C.2 Two-Stage Propagation	21
C.3 Gradients for Canonical Ray	21
C.4 Gradients for Whitening Matrix	21
C.5 Gradients for Scaling and Rotation	22
C.6 Gradients for 3D Mean	22
C.7 Numerical Stability of Our Differentiable Framework	22

864	D Appendix: Additional Details in Association	23
865	D.1 Particle Bounding Frustum (PBF) Solver	23
866	D.2 Details in Computing CSF-PBF Intersection	24
867	D.3 Relation to Prior Works	25
868		
869	E Appendix: BEAP Image Inverse Interpolation	25
870	Pinhole Camera Model	26
871	KB Fisheye Model	26
872		
873	F Appendix: High-level Algorithmic Summary	26
874		
875	G Appendix: Additional Implementation Details	26
876	Baselines	26
877	Implementation Details	26
878	Central and peripheral regions in ScanNet++ evaluation	27
879	Cross-camera generalization evaluation on ZipNeRF	28
880		
881	H Appendix: EWA Splatting Preliminaries	28
882	From camera coordinates to ray coordinates	28
883	Local affine approximation	28
884	Splatting approximation error	28
885		
886	I Appendix: Advantage over Prior Ray-Tracing Methods	29
887		
888	J Appendix: Results on Egocentric Dataset	29
889		
890	K Appendix: Additional Material and Results	31
891	K.1 Additional Material.	31
892	K.2 Additional Results.	31
893		
894		
895		
896		
897		
898		
899		
900		
901		
902		
903		
904		
905		
906		
907		
908		
909		
910		
911		
912		
913		
914		
915		
916		
917		

LLM USAGE STATEMENT

We used a large language model (ChatGPT, OpenAI) as a writing assistant. Specifically, the LLM was employed to (i) improve the clarity and readability of the manuscript, (ii) suggest stylistic edits to figure captions and section transitions, and (iii) provide alternative phrasings for technical descriptions without altering the scientific content.

The LLM was not involved in research ideation, algorithm design, data analysis, or experimental validation. All technical contributions, mathematical derivations, and results are the work of the authors. The authors take full responsibility for the content of this paper.

A APPENDIX: EXACTNESS AND LIMITATION

We state precisely what “exactness” denotes in this work: the *elimination of linear approximation in projective geometry* throughout both the rendering and the ray-particle association stages—an error that, unlike a capacity-limited one, does not diminish as the Gaussian count grows (Sec. 4.1). We do not claim exactness in the full physical-volumetric sense; ordering, self-attenuation, and overlap remain approximated, and we identify these as the boundaries of the present formulation.

Limitations. Because our formulation integrates each Gaussian independently, it does not explicitly resolve self-occlusion, and ordering is handled by depth-based sorting—which, as the supplementary videos show, can produce occasional popping artifacts. To our knowledge EVER (Mai et al. 2025) is the only method that resolves ordering and self-occlusion analytically, and it does so by approximating each Gaussian as a constant-density ellipsoid at a substantial cost in rendering speed.

This limitation is, however, largely inconsequential in practice. Prior ray-tracing-based Gaussian renderers—Fuzzy Metaballs (Keselman & Hebert 2022) and the subsequent 3DGR/UT family (Moënné-Loccoz et al. 2024; Wu et al. 2025)—all rest on the “maximum-response” assumption for ray-Gaussian interaction, which likewise ignores ordering and self-attenuation yet has been shown to introduce negligible error regardless of the Gaussian count (Celarek et al. (2025), Fig. 9–11 / Sec. 7.1). Our depth-based ordering therefore attains the same practical accuracy as existing ray-tracing-based Gaussian renderers.

Future work. Although the practical impact of ordering and self-occlusion is small in current Gaussian-based ray tracing, both fundamentally bound exact volumetric correctness, and two directions follow naturally. The first is to incorporate analytic or learnable self-attenuation models—inspired by ellipsoidal integration (e.g., EVER (Mai et al. (2025))) but without its runtime penalty. The second is to treat ordering more principledly, which should further suppress the rare popping artifacts seen under extreme configurations. Both concerns are orthogonal to our contribution on projective exactness, and resolving them would extend the framework’s robustness to large-scale and high-dynamic scenes.

B APPENDIX: RAY-GAUSSIAN TRANSMITTANCE WITH PROJECTIVE EXACTNESS

B.1 PRELIMINARIES AND NOTATIONS

We adopt the classical parameterization of 3D Gaussian primitives (Zwicker et al. 2001), in which each Gaussian is specified by a mean position $\mu \in \mathbb{R}^3$ and a 3D covariance matrix Σ in the world coordinate system:

$$\mathcal{G}_{\Sigma, \mu}(\mathbf{x}) = \frac{1}{\rho |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \right), \quad (\text{B.1})$$

where the normalization constant is $\rho = \sqrt{2\pi}$ and $\mathbf{x} \in \mathbb{R}^3$ is a point in world space. Unlike recent methods such as 3DGS (Kerbl et al. 2023) and 3DGUT (Wu et al. 2025), which drop the determinant term $|\Sigma|^{1/2}$ to simplify splatting or association, we retain it, since it is what renders the subsequent ray integral exact rather than merely proportional. The covariance factorizes as

$\Sigma = RSS^T R^T$, with a scaling matrix $S = \text{diag}(\mathbf{s})$ built from the scaling vector $\mathbf{s} \in \mathbb{R}^3$ and a rotation matrix $R \in \text{SO}(3)$ derived from a unit quaternion $\mathbf{q} \in \mathbb{R}^4$.

The transmittance contributed by a single 3D Gaussian particle along a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ is then

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\Sigma, \mu}(\mathbf{o} + t\mathbf{d}) dt, \quad (\text{B.2})$$

where σ is the opacity coefficient of the Gaussian.

Finally, for the set of depth-sorted 3D Gaussian particles contributing to a given ray, the rendered color C follows the standard front-to-back compositing rule:

$$C(\mathbf{o}, \mathbf{d}) = \sum_i c_i(\mathbf{o}, \mu_i) T_i \prod_{j=1}^{i-1} (1 - T_j), \quad (\text{B.3})$$

where $c_i(\mathbf{o}, \mu_i)$ is the view-dependent color of the i -th Gaussian seen from the optical center $\mathbf{o}$, parameterized by spherical harmonics (SH) (Kerbl et al., 2023). Minimizing the reconstruction loss between rendered and observed color along each ray jointly optimizes the parameters $\mathbf{s}$, $\mathbf{q}$, μ , the opacity coefficient σ , and the SH coefficients by backpropagation.

B.2 DIFFERENTIABLE RENDERING WITH PROJECTIVE EXACTNESS

Achieving projective-exact Gaussian rendering in a ray-marching formulation hinges on a single obstacle: obtaining a closed-form solution for the integral of Gaussian density and color along a 3D ray. We overcome it by transforming the ray—per Gaussian—into a canonical coordinate system in which the transmittance integral (Eq. B.2) is preserved and the Gaussian becomes isotropic, and then deriving the exact ray color from first principles.

Concretely, we introduce a canonical coordinate system shared across all 3D Gaussian primitives. A point $\mathbf{u} \in \mathbb{R}^3$ in this space maps to world coordinates, per Gaussian, via:

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = H \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (\text{B.4})$$

Under this transformation, an infinitesimal ray segment $\Delta \mathbf{u}$ in the canonical frame corresponds to $\Delta \mathbf{x} = RS \Delta \mathbf{u}$ in the world frame.

Preservation of the per-Gaussian transmittance across coordinate systems requires the elementary transmittance—the product of density and segment length—to be invariant under the transformation (see Fig. 2 Right):

$$\mathcal{G}_{\mathbf{I}, \mathbf{0}}(\mathbf{u}) |\Delta \mathbf{u}| = \mathcal{G}_{\Sigma, \mu}(\mathbf{x}) |\Delta \mathbf{x}|. \quad (\text{B.5})$$

This constraint absorbs the Jacobian determinant into the measure; since $|\frac{\Delta \mathbf{x}}{\Delta \mathbf{u}}| = |RS| = |\Sigma|^{1/2}$, the canonical density becomes:

$$\mathcal{G}_{\mathbf{I}, \mathbf{0}}(\mathbf{u}) = \frac{1}{\rho} \exp \left(-\frac{1}{2} \mathbf{u}^T \mathbf{u} \right), \quad (\text{B.6})$$

which is isotropic and shared by all primitives in canonical space.

Consequently, the exact closed-form transmittance integral of each Gaussian along $\mathbf{r}(t)$ is obtained by accumulating density along the transformed ray $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$ through the canonical isotropic Gaussian. The contribution from $t < 0$ is always negligible unless the optical center lies within the 3-sigma contour, a configuration that every 3D Gaussian rendering framework culls during optimization.

The integral admits a *closed-form* solution in the perpendicular distance from the ray to the Gaussian center, as stated in Theorem 1

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{0}}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp \left(-\frac{1}{2} (D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2 \right), \quad (\text{B.7})$$

where $D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d})$ is the perpendicular distance from the Gaussian center (the canonical-space origin) to the transformed ray $\mathbf{r}_u(t)$. Geometrically, this is equivalently the minimal *Mahalanobis*

distance from any point on the ray to the Gaussian in world space—a direct measure of how closely the ray passes the particle distribution.

It remains only to compute this perpendicular distance from the canonical origin to the transformed ray. We express the transformed ray’s direction vector $\mathbf{d}_u$ and moment vector $\mathbf{m}_u$ as:

$$\mathbf{d}_u = (S^{-1}R^\top) \mathbf{d}, \quad \mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u, \quad (\text{B.8})$$

where $\mathbf{o}_u = (S^{-1}R^\top) (\mathbf{o} - \boldsymbol{\mu})$ is the optical center in canonical space. The squared distance $D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d})$ then follows efficiently as:

$$(D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d}))^2 = \frac{\mathbf{m}_u^\top \mathbf{m}_u}{\mathbf{d}_u^\top \mathbf{d}_u}. \quad (\text{B.9})$$

Substituting this squared distance into Eq. B.7 and the resulting transmittance into Eq. B.3 yields the accumulated color along the ray.

B.3 RAY INTEGRAL OF ISOTROPIC 3D GAUSSIAN

Theorem 1. *Given a ray and a standard isotropic 3D Gaussian, the ray-Gaussian integral has a closed-form proportional to:*

$$\exp\left(-\frac{1}{2}D^2\right),$$

where D is the perpendicular distance from the Gaussian center to the ray.

Proof. We first translate the coordinate system so that the Gaussian $\mathcal{G}$ is centered at the origin, translating the ray $\mathbf{r}_u$ accordingly, and then rotate the scene so that the ray direction aligns with the z -axis. By the translation-invariance of the integral and the isotropy of the standard Gaussian, neither transformation alters its value.

The translated standard isotropic 3D Gaussian is:

$$\mathcal{G}_{\mathbf{I},\mathbf{o}}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2}\mathbf{u}^\top \mathbf{u}\right), \quad (\text{B.10})$$

which matches Eq. B.6. Let the z -axis aligned ray as:

$$\mathbf{r}_u(t) = \begin{bmatrix} \mathbf{o}_u \\ 0 \end{bmatrix} + t \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad (\text{B.11})$$

the ray-Gaussian integral becomes:

$$\begin{aligned} \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I},\mathbf{o}}(\mathbf{r}_u(t)) dt &= \frac{1}{\rho} \exp\left(-\frac{u_0^2 + v_0^2}{2}\right) \int_{t \in \mathbb{R}} \exp\left(-\frac{1}{2}t^2\right) dt \\ &= \frac{\sqrt{2\pi}}{\rho} \exp\left(-\frac{1}{2}D^2\right), \end{aligned} \quad (\text{B.12})$$

where $D^2 = \frac{u_0^2 + v_0^2}{2}$ denotes the squared perpendicular distance from the ray to the Gaussian center in canonical space. Thus, the integral is proportional to $\exp(-\frac{1}{2}D^2)$, as stated in theorem. $\square$

The density function shared by our canonical-space primitives (Eq. B.6) coincides with the isotropic form used above. Although D takes the world-coordinate ray $(\mathbf{o}, \mathbf{d})$ as input, its value also depends on the world-space Gaussian parameters $\boldsymbol{\mu}, \Sigma$, which define the canonical transformation of the ray. Hence, for each Gaussian contributing to the ray, the transmittance is:

$$T = \frac{\sqrt{2\pi}\sigma}{\rho} \exp\left(-\frac{1}{2}(D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.13})$$

which recovers Eq. B.7 for the choice $\rho = \sqrt{2\pi}$.

C APPENDIX: COOKBOOK OF GRADIENT COMPUTATION

C.1 CHAIN RULE

Recall that $T = \sigma\alpha$ is the ray–Gaussian transmittance (Eq. B.7), with σ the opacity coefficient and α the ray–Gaussian integral. Let $\kappa = D_{\mu, \Sigma}^2$ denote the minimal squared Mahalanobis distance from the ray to the Gaussian, S the Gaussian scaling, and R the world-space rotation matrix. Given the world-space 3D covariance $\Sigma = RSS^\top R^\top$, the PCA whitening matrix W_{pca} satisfies:

$$W_{\text{pca}} = \Sigma^{-\frac{1}{2}} = S^{-1}R^\top, \quad (C.1)$$

where $S^{-1} = \text{diag}\left(\frac{1}{s}\right).$

Following the gradient flow in Fig. 2 the chain rule gives the derivatives *w.r.t.* the Gaussian parameters—the scaling vector $\mathbf{s}$, rotation quaternion $\mathbf{q}$, and mean μ :

$$\frac{\partial T}{\partial \mathbf{s}} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{s}}, \quad (C.2)$$

$$\frac{\partial T}{\partial \mathbf{q}} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{q}}, \quad (C.3)$$

$$\frac{\partial T}{\partial \mu} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial \mu}. \quad (C.4)$$

C.2 TWO-STAGE PROPAGATION

To minimize the overhead of shared propagation, we structure gradient computation as the two-stage pipeline of 3DGS (Kerbl et al., 2023), comprising `RenderCUDA` (RC) and `PreprocessCUDA` (PC). RC performs ray-wise accumulation, accelerated by our Particle Bounding Frustum (PBF) association (Sec. 3.2), whereas PC operates at Gaussian-wise complexity and gains additional speedup from view-frustum culling.

From Eq. B.7 the gradient *w.r.t.* the opacity coefficient and the partial derivative shared across Gaussian parameters, both evaluated in RC, are:

$$\left. \frac{\partial T}{\partial \sigma} \right|_{\text{RC}} = \alpha, \quad \left. \frac{\partial T}{\partial \kappa} \right|_{\text{RC}} = -\frac{T}{2}. \quad (C.5)$$

The remaining parameter gradients then propagate from the squared distance, namely $\frac{\partial \kappa}{\partial \mathbf{s}}, \frac{\partial \kappa}{\partial \mathbf{q}}, \frac{\partial \kappa}{\partial \mu}$.

C.3 GRADIENTS FOR CANONICAL RAY

We next backpropagate to the canonical ray parameters—the origin $\mathbf{o}_u$, direction $\mathbf{d}_u$, and moment $\mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u$ of Eq. B.8. These quantities enter the Mahalanobis distance in Eq. B.9 directly and encode the canonical ray geometry.

$$\left. \frac{\partial \kappa}{\partial \mathbf{m}_u} \right|_{\text{RC}} = \left(\frac{2}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{m}_u, \quad (C.6)$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{o}_u} \right|_{\text{RC}} = \frac{\partial \kappa}{\partial \mathbf{m}_u} \times \mathbf{d}_u, \quad (C.7)$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{d}_u} \right|_{\text{RC}} = - \left(\frac{2\kappa}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{d}_u + \mathbf{o}_u \times \frac{\partial \kappa}{\partial \mathbf{m}_u}. \quad (C.8)$$

C.4 GRADIENTS FOR WHITENING MATRIX

Recall that $\mathbf{d}_u = W_{\text{pca}} \mathbf{d}$ and $\mathbf{o}_u = W_{\text{pca}}(\mathbf{o} - \mu)$, where $\mathbf{o} = -R_c^\top \mathbf{t}_c$ is the world-space optical center and $\mathbf{d}$ the ray direction. Given the ray-parameter gradients, we propagate them to the PCA whitening matrix W_{pca} through outer products with the corresponding world-space vectors:

$$\begin{aligned}
\text{RenderCUDA: } \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \Big|_{\text{RC}} &= \frac{\partial \kappa}{\partial \mathbf{d}_u} \mathbf{d}^\top; \\
\text{PreprocessCUDA: } \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \Big|_{\text{PC}} &= \frac{\partial \kappa}{\partial \mathbf{o}_u} (\mathbf{o} - \boldsymbol{\mu})^\top; \\
\text{Overall: } \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} &= \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \Big|_{\text{RC}} + \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \Big|_{\text{PC}}.
\end{aligned} \tag{C.9}$$

The gradient through $\mathbf{d}_u$ is accumulated ray-wise in `RenderCUDA`, while that through $\mathbf{o}_u$ is computed Gaussian-wise in `PreprocessCUDA`; summing the two yields the complete gradient *w.r.t.* the PCA transformation matrix $\mathbf{W}_{\text{pca}}$.

C.5 GRADIENTS FOR SCALING AND ROTATION

Thus, for scaling gradients $\frac{\partial \kappa}{\partial \mathbf{s}} = \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \frac{\partial \mathbf{W}_{\text{pca}}}{\partial \mathbf{s}}$, we have:

$$\frac{\partial \mathbf{W}_{\text{pca}}^{i,j}}{\partial s_k} \Big|_{\text{PC}} = \begin{cases} -\frac{1}{s_k^2} R_{j,k} & \text{if } i = k \\ 0 & \text{otherwise} \end{cases}. \tag{C.10}$$

To derive gradients for rotation $\frac{\partial \kappa}{\partial \mathbf{q}} = \frac{\partial \kappa}{\partial \mathbf{W}_{\text{pca}}} \frac{\partial \mathbf{W}_{\text{pca}}}{\partial \mathbf{q}}$, given the quaternion form $\mathbf{q} = q_r + q_i \mathbf{i} + q_j \mathbf{j} + q_k \mathbf{k}$, we can write the whitening matrix as:

$$\mathbf{W}_{\text{pca}} = 2 \begin{pmatrix} \frac{1}{s_1}(\frac{1}{2} - q_j^2 - q_k^2) & \frac{1}{s_1}(q_i q_j + q_r q_k) & \frac{1}{s_1}(q_i q_k - q_r q_j) \\ \frac{1}{s_2}(q_i q_j - q_r q_k) & \frac{1}{s_2}(\frac{1}{2} - q_i^2 - q_k^2) & \frac{1}{s_2}(q_j q_k + q_r q_i) \\ \frac{1}{s_3}(q_i q_k + q_r q_j) & \frac{1}{s_3}(q_j q_k - q_r q_i) & \frac{1}{s_3}(\frac{1}{2} - q_i^2 - q_j^2) \end{pmatrix}. \tag{C.11}$$

As a result, we derive the following end gradients for the quaternion $\mathbf{q}$ as:

$$\begin{aligned}
\frac{\partial \mathbf{W}_{\text{pca}}}{\partial q_r} \Big|_{\text{PC}} &= 2 \begin{pmatrix} 0 & \frac{q_k}{s_1} & -\frac{q_j}{s_1} \\ -\frac{q_k}{s_2} & 0 & \frac{q_i}{s_2} \\ \frac{q_j}{s_3} & -\frac{q_i}{s_3} & 0 \end{pmatrix}, \quad \frac{\partial \mathbf{W}_{\text{pca}}}{\partial q_i} \Big|_{\text{PC}} = 2 \begin{pmatrix} 0 & \frac{q_j}{s_2} & \frac{q_k}{s_2} \\ \frac{q_k}{s_3} & -\frac{q_r}{s_3} & -\frac{2q_i}{s_3} \\ -\frac{2q_j}{s_3} & \frac{q_i}{s_3} & -\frac{q_r}{s_3} \end{pmatrix}, \\
\frac{\partial \mathbf{W}_{\text{pca}}}{\partial q_j} \Big|_{\text{PC}} &= 2 \begin{pmatrix} -\frac{2q_j}{s_1} & \frac{q_i}{s_1} & -\frac{q_r}{s_1} \\ \frac{q_i}{s_2} & 0 & \frac{q_k}{s_2} \\ \frac{q_r}{s_3} & \frac{q_k}{s_3} & -\frac{2q_j}{s_3} \end{pmatrix}, \quad \frac{\partial \mathbf{W}_{\text{pca}}}{\partial q_k} \Big|_{\text{PC}} = 2 \begin{pmatrix} -\frac{2q_k}{s_1} & \frac{q_r}{s_1} & \frac{q_i}{s_1} \\ -\frac{q_r}{s_2} & -\frac{2q_k}{s_2} & \frac{q_j}{s_2} \\ \frac{q_i}{s_3} & \frac{q_j}{s_3} & 0 \end{pmatrix}.
\end{aligned} \tag{C.12}$$

C.6 GRADIENTS FOR 3D MEAN

Finally, the end gradient *w.r.t.* the Gaussian 3D mean $\boldsymbol{\mu}$ is:

$$\frac{\partial \kappa}{\partial \boldsymbol{\mu}} \Big|_{\text{PC}} = -\mathbf{W}_{\text{pca}}^\top \frac{\partial \kappa}{\partial \mathbf{o}_u}. \tag{C.13}$$

C.7 NUMERICAL STABILITY OF OUR DIFFERENTIABLE FRAMEWORK

The choice of forward formulation, backward gradient computation, and intermediate cache variables is decisive for numerical stability. Prior approaches such as GOF (Yu et al. 2024b) are unstable in backward propagation and consequently depend on auxiliary 3D filtering heuristics to keep training well-behaved. Our closed-form formulation and its exact gradient chain avoid this instability by construction, dispensing with any such filtering.

The decisive difference lies in how $\mathbf{m}_u^\top \mathbf{m}_u$ is expanded. In exact arithmetic, the two forms are geometrically identical:

$$\mathbf{m}_u^\top \mathbf{m}_u = \|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2. \tag{C.14}$$

The right-hand form, however, is fragile: when one axis of a 3D Gaussian is scaled toward zero, the canonical-space vectors $\mathbf{o}_u$ and $\mathbf{d}_u$ become nearly parallel, so that $\cos(\angle(\mathbf{o}_u, \mathbf{d}_u)) \rightarrow 1$ while both



Figure C.1: **Artifacts Without 3D Filtering in GOF.** (Left) GOF with degenerate Gaussians left unfiltered; (Right) the stable result obtained by substituting our forward and backward formulations.

$\|\mathbf{o}_u\|$ and $\|\mathbf{d}_u\|$ diverge. The difference $\|\mathbf{o}_u\|^2\|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2$ is then a catastrophic cancellation—prone to overflow and, in practice, capable of returning negative values that in turn destabilize the backward pass.

Fig. C.1 (Left) shows the resulting artifacts in GOF when degenerate Gaussians are not filtered out: the snowflake noise originates in the overflow of Eq. C.14 where prematurely negative values and vanishingly small positive values inflate the Gaussian intensity. Because the EWA approximation with Jacobian and conic is itself numerically stable, these artifacts trace specifically to the inaccurate maximum-response computation, which additionally produces the gridline pattern.

Our CUDA rasterizer instead evaluates the cross product and its gradients directly (Eq. B.9 forward; Eqs. C.6–C.9 backward), sidestepping the cancellation entirely: the forward pass is artifact-free and the backward pass returns accurate gradients. Fig. C.1 (Right) confirms this—substituting our forward and backward formulations into GOF removes the instability-induced artifacts even without any 3D filtering of degenerate (flat disk-like or needle) Gaussians.

D APPENDIX: ADDITIONAL DETAILS IN ASSOCIATION

Section D.1 gives the detailed derivation, Sec. D.2 the implementation considerations that keep the PBF stable under large-FoV and omnidirectional cameras, and Sec. D.3 the derivational relationships together with the exact conditions under which prior formulas reduce to ours.

D.1 PARTICLE BOUNDING FRUSTUM (PBF) SOLVER

Recalling Eq. 8, let $P = (VH)^\top \in \mathbb{R}^{4 \times 4}$ be the plane transformation matrix from view space to canonical space. Taking the unknown bounds $c = \tan \theta_{1,2}$ as the representative case:

$$\mathbf{g}_u = -P_0 + cP_2 = \begin{bmatrix} -P_{00} + cP_{02} \\ -P_{10} + cP_{12} \\ -P_{20} + cP_{22} \\ -P_{30} + cP_{32} \end{bmatrix}, \quad (\text{D.1})$$

where P_i is the i -th row of the matrix, written as a 4×1 column vector, and P_{ij} its j -th element. Recalling Eq. 9 and taking the 1-standard-deviation contour ellipsoid of the Gaussian ($\lambda = 1$) for concreteness, the tangency condition becomes the quadratic:

$$((\mathbf{f} \circ P_2^\top) \cdot P_2) c^2 - 2((\mathbf{f} \circ P_0^\top) \cdot P_2) c + (\mathbf{f} \circ P_0^\top) \cdot P_0 = 0, \quad (\text{D.2})$$

where $\mathbf{f} = (1, 1, 1, -1)$ is a 1×4 row vector. We abbreviate this as:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad (\text{D.3})$$

where $\mathcal{T}$ is the 3×3 upper-triangular matrix with entries $\mathcal{T}_{ij} = (\mathbf{f} \circ P_i^\top) \cdot P_j$.

Similarly, for the bounds of ϕ , we have $c = \tan \phi_{1,2}$ satisfying:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{12}c + \mathcal{T}_{11} = 0. \quad (\text{D.4})$$

Notably, for a camera-view configuration $[R_c \mid \mathbf{t}_c]$, the parameter matrix $\mathcal{T}$ is computed directly from the Gaussian’s view-space covariance $\Sigma_c = R_c R S S^\top R_c^\top$ and 3D mean $\mu_c = R_c \mu + \mathbf{t}_c$:

$$\mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (\text{D.5})$$

The angular bounds—expressed through their tangent-space center and extent—then follow as:

$$\begin{aligned} \left(\frac{\tan \theta_1 + \tan \theta_2}{2}, \frac{\tan \phi_1 + \tan \phi_2}{2} \right) &= \left(\frac{\mathcal{T}_{02}}{\mathcal{T}_{22}}, \frac{\mathcal{T}_{12}}{\mathcal{T}_{22}} \right), \\ \|\tan \theta_1 - \tan \theta_2\| &= \frac{2\sqrt{\mathcal{T}_{02}^2 - \mathcal{T}_{22}\mathcal{T}_{00}}}{|\mathcal{T}_{22}|}, \\ \|\tan \phi_1 - \tan \phi_2\| &= \frac{2\sqrt{\mathcal{T}_{12}^2 - \mathcal{T}_{22}\mathcal{T}_{11}}}{|\mathcal{T}_{22}|}. \end{aligned} \quad (\text{D.6})$$

Should the discriminant of either quadratic turn negative—the signature of an optical center lying inside the Gaussian ellipsoid, where the angular bounds are undefined—we conservatively clamp the frustum angles to the camera’s maximum field of view (FoV).

Crucially, this result is computed directly from the true 3D view-space covariance, whereas EWA and UT route through an intermediate screen-space conic approximation; the PBF formulation is thus exact and efficient at once.

D.2 DETAILS IN COMPUTING CSF-PBF INTERSECTION

We sample spherical angles uniformly within the maximum FoV. The resulting x, y, z coordinates—the ray direction vectors $\mathbf{d}$ —are projected onto the unit sphere and mapped into pixel space for inverse color interpolation (Sec. E), and the sampled rays are grouped by their CSFs for the subsequent Gaussian association.

For pinhole cameras of limited FoV, the PBF angular bounds ordinarily lie within $[-\frac{\pi}{2}, \frac{\pi}{2}]$, where $\tan$ is monotonically increasing; PBF-CSF intersections can then be decided simply by comparing the $\tan$ values of their bounds. Under wide FoVs, however, the PBF bounds may fall outside this monotonic interval even when the Gaussian-center incidence angle remains within $[-\frac{\pi}{2}, \frac{\pi}{2}]$, so a direct comparison of $\tan$ values can yield incorrect intersections—for instance, spuriously matching the bounds as $[\theta'_2, \theta_1]$ (Fig. D.1).

To recover the correct intersections, we apply a mirror transformation to the tangent values of both the PBF angular bounds and the CSF partitions. Following the CMEI camera model (Mei & Rives 2007), we shift the optical center by a mirror parameter ξ , yielding:

$$\tan \theta_\xi = \frac{\tan \theta}{1 + \xi \frac{z}{\|\mathbf{z}\|} \sqrt{1 + \tan^2 \theta}} = \frac{\sin \theta}{\cos \theta + \xi}, \quad (\text{D.7})$$

where θ_ξ is the mirror-transformed angle of θ on the xz -plane under mirror parameter ξ ; we fix $\xi = 1$ throughout.

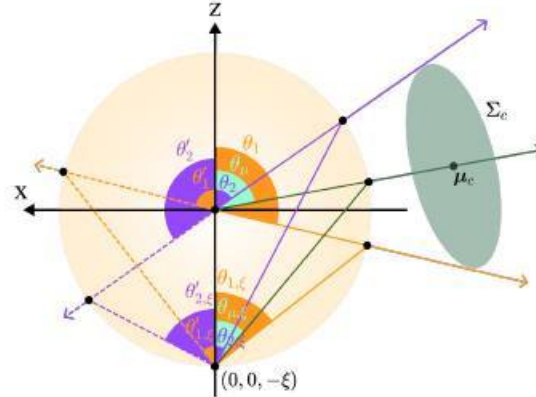


Figure D.1: Mirror Transformation of PBF Angular Bounds. A given pair of bounds generates four candidate mirrored angles $\theta_{1,\xi}, \theta'_{1,\xi}, \theta_{2,\xi}, \theta'_{2,\xi}$. For a Gaussian centered at μ_c , the two candidates closest to the Gaussian’s own mirrored angle $\theta_{\mu,\xi}$ serve as the PBF bounds, ensuring robust intersection with mirror-transformed CSF partitions.

For a pair of tangent values $[\tan \theta_1, \tan \theta_2]$ returned by the PBF solver (Eq. D.6), this transformation produces four candidate mirror angles (Fig. D.1), whereas the Gaussian center maps to a single mirror-transformed angle $\theta_{\mu,\xi}$. Selecting the two candidates nearest $\theta_{\mu,\xi}$ fixes the effective lower and upper PBF bounds $[\theta_{1,\xi}, \theta_{2,\xi}]$, which are then compared against the CSF partitions to compute the correct intersections.

This construction keeps angular comparisons robust and consistent even beyond the monotonic domain of $\tan$, and it is cheap: the timing analysis (Tab. K.5) shows that the entire pre-processing pipeline—mirror-angle-based PBF-CSF intersection plus the canonical transformation of all Gaussians—completes in under 0.13 ms, confirming efficiency even for thousands of Gaussians in wide-FoV scenes.

D.3 RELATION TO PRIOR WORKS

Substituting the perspective screen space coordinates:

$$u = \tan \theta, \quad v = \tan \phi$$

into our formulation reproduces exactly the expressions of 2DGS (Huang et al. 2024a) and HTGS (Hahlbohm et al. 2025)—as expected, since those correspond to the perspective specialization of our angular-domain formulation.

The distinction is one of starting assumptions. 2DGS and HTGS parameterize in screen space or on a tangent plane and therefore invoke geometric identities valid only for perspective (pinhole) cameras—those that arise when rays are implicitly tied to a planar, z -parallel image plane; the identities do not carry over to fisheye or omnidirectional systems, where the corresponding screen-space surface is curved. Our derivation instead begins from a camera-agnostic angular parameterization in 3D and directly parameterizes Camera Sub-Frustums (CSFs) and Particle Bounding Frustums (PBFs). Because the two derivations start from different known quantities and constraints, they differ precisely in the conditions under which closed-form bounds and culling guarantees remain valid for generic cameras.

We emphasize that mathematics is a general tool; the contribution here lies in deriving these bounds under weaker assumptions—free of any screen-space structure—and in validating them experimentally across both pinhole and wide-FoV camera families.

This difference in parameterization propagates through the downstream pipeline. Association, tiling/duplication, and sorting all depend on how Gaussians map to tiles or bins, and an angular (CSF-based) domain induces a bucketing structure distinct from pinhole screen-space tiling. The distinction is consequential at large FoVs: under generic cameras a Gaussian’s bound can appear partly “behind” the camera in screen coordinates (Fig. D.1), leading screen-space methods to mirrored or incorrect tangent-based bounds (dashed line in Fig. D.1). We handle these cases robustly by introducing mirror parameters in the association step (Sec. D.2) and folding the Gaussian center into sorting and correction—steps that a perspective-only derivation never needs.

E APPENDIX: BEAP IMAGE INVERSE INTERPOLATION

To prepare training data, we sample rays that cover the full FoV of the original image and pass them as input to our CUDA renderer. In parallel, we interpolate the ground-truth color for each input ray, so that supervision is both accurate and uniform across the FoV.

Projecting the sampled rays onto the observed images begins by converting the incidence-angle projections θ and ϕ into normalized ray direction vectors $\mathbf{d} = (x, y, z)^\top$:

$$x = \frac{\sin \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.1})$$

$$y = \frac{\cos \theta \sin \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.2})$$

$$z = \frac{\cos \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}. \quad (\text{E.3})$$

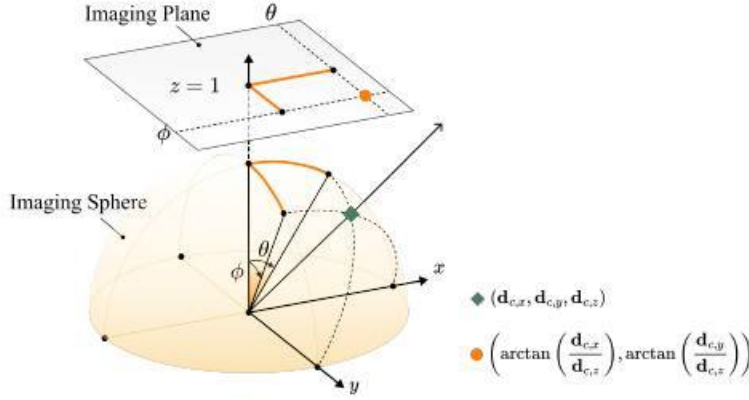


Figure E.1: **Spherical Angles Defining PBF and BEAP.** A view-space direction $\mathbf{d}_c$ is normalized onto the optical sphere (green spade) and then projected angularly onto the xz and yz planes; the orange point marks the resulting coordinate (θ, ϕ) on the unfolded BEAP imaging plane.

We support several camera models for the subsequent projection of the normalized rays $(x, y, z)^\top$ into pixel space $(x_p, y_p, 1)^\top$.

Pinhole Camera Model. For the pinhole model, $\frac{x}{z} = \tan \theta$ and $\frac{y}{z} = \tan \phi$, so that:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.4})$$

KB Fisheye Model. The incidence angle φ follows from its projections:

$$r = \sqrt{\tan^2 \theta + \tan^2 \phi} \quad \text{and} \quad \varphi = \arctan(r). \quad (\text{E.5})$$

and the fisheye-distorted angle is:

$$\varphi_d = \varphi(1 + k_1\varphi^2 + k_2\varphi^4 + k_3\varphi^6 + k_4\varphi^8). \quad (\text{E.6})$$

The rays are thereby projected into pixel space, enabling inverse color interpolation from:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{f_x \varphi_d}{r} & 0 & c_x \\ 0 & \frac{f_y \varphi_d}{r} & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.7})$$

F APPENDIX: HIGH-LEVEL ALGORITHMIC SUMMARY

We summarize the volumetric rendering pipeline at a high level in Alg. 1 and Alg. 2. To make the differences legible, we mirror the structure of 3DGS (Kerbl et al., 2023): newly introduced or replaced modules are marked in green, and substantially modified modules in yellow.

G APPENDIX: ADDITIONAL IMPLEMENTATION DETAILS

Baselines. We compare 3DGEER against the leading volumetric particle rendering methods that target the projective approximation error of 3DGS (Kerbl et al., 2023). For large-FoV fisheye inputs, the baselines are FisheyeGS (Liao et al., 2024), EVER (Mai et al., 2025), and 3DGUT (Wu et al., 2025); under pinhole settings we additionally include 3DGS and 3DGRT (Moënné-Loccoz et al., 2024).

ALGORITHM 1: GPU Software Asso. & Render.

Function 3DGEER ($\mu, s, q, c, \sigma, r_c, V$) :

- CullGaussian(μ, V) ▷ Frustum Culling
- $F \leftarrow \text{InitCSFs}(w, h, K)$ ▷ Partition
- $\Sigma_c, \mu_c \leftarrow \text{ViewspaceGaussians}(s, q, V)$ ▷ Transform
- $F_{ID} \leftarrow \text{PBF_Intersect}(\Sigma_c, \mu, F)$ ▷ Sub-frustum Asso.
- $W_{pca} \leftarrow \text{PCA_Whitening}(s, q)$ ▷ Transform
- $\mathcal{G}_{ID}, F_K \leftarrow \text{DuplicateWithKeys}(W_{pca}, F_{ID})$ ▷ Indices (Gaussian) & Keys (FrustumID+Depth)
- $\text{SortByKey}(\mathcal{G}_{ID}, F_K)$ ▷ Globally Depth Sort
- $R \leftarrow \text{IdentifyFrustumRanges}(F, F_K)$
- $I \leftarrow 0$ ▷ Init Canvas (w/ Rays)
- for all** CSFs $f \leftarrow I$ **do**
- for all** Rays $i \leftarrow f$ **do**
- $r \leftarrow \text{GetFrustumRange}(R, f)$
- $m_u, d_u \leftarrow \text{Cano_Ray}(W_{pca}, \mu, i)$ ▷ Transform
- $D_{\mu, \Sigma} \leftarrow \text{Dist}(m_u, d_u)$ ▷ Mah-distance
- $I[i] \leftarrow \text{BlendInOrder}(i, \mathcal{G}_{ID}, r, F_K, D_{\mu, \Sigma}, c, \sigma)$ ▷ Alpha Blending
- end**
- end**
- return** I

ALGORITHM 2: 3DGEER Initialization and Training Framework

w, h : width and height of the training images
 K, V : intrinsics and extrinsic meta info.

$\mu \leftarrow \text{SfM Points}$ ▷ Init Gaussian Attributes

- $s, q \leftarrow \text{InitAttributes}()$ ▷ Position
- $c, \sigma \leftarrow \text{InitCoefficients}()$ ▷ Scaling, Rotation
- $c, \sigma \leftarrow \text{InitCoefficients}()$ ▷ Color, Opacity

while not converged **do**

- $r_c, \hat{C} \leftarrow \text{BEAP}(w, h, K)$ ▷ View Rays / Colors
- $C \leftarrow 3DGEER(\mu, s, q, c, \sigma, r_c, V)$ ▷ Asso. & Render
- $\mathcal{L} \leftarrow \text{Loss}(C, \hat{C})$ ▷ See Alg. 1
- $\mu, s, q, c, \sigma \leftarrow \text{Adam}(\nabla \mathcal{L})$ ▷ Loss
- if** IsRefinementIteration(i) **then** ▷ Backprop & Step
- $\mu, s, q, c, \sigma \leftarrow \text{Optimize}(\mu, s, q, c, \sigma)$ ▷ Kerbl et al. (2023)
- end**

end

Implementation Details. For a fair comparison, we align our implementation with the original 3DGS in initialization and hyperparameters on ScanNet++ and MipNeRF360, and adopt the EVER and SMERF (Duckworth et al., 2024) training configurations for all candidate methods on ZipNeRF. Like most ray-based frameworks, our method does not access gradients of mean2D for densification or pruning; it instead optimizes using gradients of the view-space mean3D. Unless otherwise noted, all models share consistent supervision strategies and architectural settings. On the fisheye datasets (ScanNet++ and ZipNeRF) we precompute bijective grid mappings (Guo et al., 2025) and reproject every rendered output to the native KB-Fisheye image space (Kannala & Brandt, 2006) prior to evaluation. ScanNet++ and MipNeRF360 experiments use full-resolution images; for ZipNeRF, given its high frame count and very large fisheye resolution, we use the maximum resolution (Tab. 3) at which all baselines run reliably on a local desktop.

Central and peripheral regions in ScanNet++ evaluation. The rays sampled in the central part follows the range:

$$\theta \in [-\frac{w}{2f_x}, \frac{w}{2f_x}], \phi \in [-\frac{h}{2f_y}, \frac{h}{2f_y}].$$

The peripheral part comprises the rays sampled over the remaining angular range. Here w, h are the image width and height, and f_x, f_y the focal lengths of the original KB camera model.

Cross-camera generalization evaluation on ZipNeRF. ZipNeRF supplies two data sets: raw fisheye images and their undistorted counterparts, the latter produced by distortion correction and central cropping. Although the frames are aligned per frame, the camera poses and point clouds were reconstructed independently with COLMAP (Schönberger & Frahm, 2016) for each set, and because COLMAP’s global optimization refines each trajectory separately, no single rigid transformation aligns the two pose sets exactly—applying one would introduce alignment error. To keep the cross-camera evaluation free of such structure-from-motion (SfM) error, we always reuse the training-time camera poses at test time, varying only the intrinsic parameters while holding the extrinsics fixed. This yields fair comparisons that are not confounded by preprocessing-induced pose inconsistencies.

H APPENDIX: EWA SPLATTING PRELIMINARIES

From camera coordinates to ray coordinates. Following the coordinate conventions of EWA Splatting (Zwicker et al., 2001), the projective transformation for pinhole cameras maps camera- (view-)space points $\mathbf{t}$ to ray-space points $\mathbf{x}$ through $\tau: \mathbf{t} \xrightarrow{\sim} \mathbf{x}$:

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \tau(\mathbf{t}) = \begin{bmatrix} t_0/t_2 \\ t_1/t_2 \\ \|(t_0, t_1, t_2)^\top\| \end{bmatrix}, \quad (\text{H.1})$$

$$\mathbf{t} = \begin{bmatrix} t_0 \\ t_1 \\ t_2 \end{bmatrix} = \tau^{-1}(\mathbf{x}) = \begin{bmatrix} x_0/l \cdot x_2 \\ x_1/l \cdot x_2 \\ 1/l \cdot x_2 \end{bmatrix}, \quad (\text{H.2})$$

$$\text{, where } l = \|(x_0, x_1, 1)^\top\|. \quad (\text{H.3})$$

These mappings are non-affine, so evaluating the ray-space integral demands either computationally expensive solutions (Westover 1990; Mao et al., 1995) or local approximations (Zwicker et al., 2001; Kerbl et al., 2023).

Local affine approximation. EWA splatting (Zwicker et al., 2001) introduces a local affine approximation τ_k that retains only the linear effect and thereby keeps the k -indexed reconstruction kernels Gaussian in 2D screen space. Defined by the first two terms of the Taylor expansion of τ at the camera-space point $\mathbf{t}_k$, it estimates a ray-space point $\mathbf{x}$ in the neighborhood of $\mathbf{x}_k$ by:

$$\mathbf{x} = \tau(\mathbf{t}) = \mathbf{x}_k + \mathbf{J}_\tau^{\mathbf{t}_k}(\mathbf{t} - \mathbf{t}_k), \quad (\text{H.4})$$

where $\mathbf{J}_\tau^{\mathbf{t}_k}$ is the Jacobian of τ at $\mathbf{t}_k$. Under this local affine transformation, the k -indexed 3D Gaussian $\mathcal{G}_{\Sigma, \mu}$ in a given camera coordinate is approximated by a 3D Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}$ in the ray coordinate. For $\forall \mathbf{x}$ around $\mathbf{x}_k$, we have:

$$\mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{x}_k)^\top \Sigma_k^{-1}(\mathbf{x} - \mathbf{x}_k)\right), \quad (\text{H.5})$$

$$\text{where } \Sigma_k = \mathbf{J}_\tau^{\mathbf{t}_k} \Sigma (\mathbf{J}_\tau^{\mathbf{t}_k})^\top. \quad (\text{H.6})$$

Integrating a 3D Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}$ along one coordinate axis yields a 2D Gaussian $\mathcal{G}_{\hat{\Sigma}_k, \hat{\mathbf{x}}_k}^{(2)}$, hence:

$$\int_{\mathbb{R}} \mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}(\mathbf{x}) dx_2 = \mathcal{G}_{\hat{\Sigma}_k, \hat{\mathbf{x}}_k}^{(2)}(\hat{\mathbf{x}}), \quad (\text{H.7})$$

where $\hat{\mathbf{x}} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$, and the mean $\hat{\mathbf{x}}_k = \begin{bmatrix} x_{k,0} \\ x_{k,1} \end{bmatrix}$, and the 2×2 variance matrix $\hat{\Sigma}_k$ is easily obtained from the 3×3 matrix Σ_k by skipping the third row and column.



Figure I.1: **Qualitative Results on the Stump (Mip-NeRF360).** 3DGEER outperforms both 3DGS-based (Gaussian Splatting) and 3DPRT-based (Particle Ray Tracing) methods on novel view synthesis benchmarks under (Left) MipNeRF360 (pinhole), while maintaining high runtime efficiency. Larger differences are visible in the colored boxes.



Figure J.1: **Qualitative Results on the Aria Dataset.** Our method produces sharper floor textures in indoor scenes and clearer distant tree details in outdoor scenes.

Splatting approximation error. Although we can not expect the density function in ray space still obey the Gaussian distribution when we apply general projective transformation to the original view space, exact density can be derived through substitution via Eq. H.2

$$\begin{aligned} \mathcal{G}_{\Sigma, \mu}(\mathbf{t}) &= \mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) \\ &= \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)^\top \Sigma_k^{-1} (\tau^{-1}(\mathbf{x}) - \mathbf{t}_k) \right), \end{aligned} \quad (\text{H.8})$$

which means that the approximation error can be estimated through Monte Carlo integration of the following equation:

$$\mathcal{E}_k = \int_{\mathbb{R}} |\mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) - \mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x})| dx_2. \quad (\text{H.9})$$

Note that x_2 represents the dimension along the ray direction in the ray coordinate.

I APPENDIX: ADVANTAGE OVER PRIOR RAY-TRACING METHODS

Pure ray-tracing (e.g., 3DGRT) can be ideally projective-exact, but in practice the BVH adopted in 3DGRT for intersection tests uses inscribed proxy icosahedron for faster speed, which is tight at vertices but face centers lie $\sim 20\%$ inward based on icosahedron’s property. Proxy shapes may degrade intersection accuracy and further give worse accumulated gradients during density control and eventually lead to incorrect geometry (e.g., missing leaves in Fig. I.1). Further, intersection tests for far, tiny, and cluttered Gaussians are limited by floating-point precision that causes noisy rendering. (E.g., in Fig. 15 of the 3DGRT paper, native rendering in the leftmost block contains visible speckles in the tree and background areas.) Even 3DGUT with UT approximation outperforms 3DGRT on MipNeRF360. For validation, we use 3DGRT’s ray-tracing and rendering engine and our 3DGEER rendering pipeline to render from the same pretrained MipNeRF360-stump and compare on PSNR $\uparrow$ /SSIM $\uparrow$ /LPIPS $\downarrow$: 3DGRT- 25.79/ 0.745/ 0.264; ours- 26.83/ 0.774/ 0.246 (+1.04/ +0.029/ -0.018). 3DGEER builds a projective-exact and efficient pipeline and further optimizes with BEAP color supervision, achieving much faster speed and better accuracy (Tab. 4).

J APPENDIX: RESULTS ON EGOCENTRIC DATASET

Recent egocentric devices typically feature wide FoVs, but existing pipelines struggle to maintain reconstruction quality when using full-resolution images or when restricted to cropped views. The

Table J.1: **Quantitative comparison on Aria**. Higher PSNR/SSIM and lower LPIPS are better.

Method	Scene		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3DGEER (Ours)	livingroom	(indoor)	36.398	0.933	0.226
	steakhouse	(outdoor)	30.662	0.881	0.246
FisheyeGS	livingroom	(indoor)	35.686	0.927	0.231
	steakhouse	(outdoor)	29.989	0.869	0.261

challenge is further compounded by low-light imaging conditions—where limited light intake, especially with small sensors and aperture-constrained lenses, exacerbates degradation.

We evaluate on the Aria egocentric dataset ($\sim 110^\circ$ FoV, circular shape). Our method achieves an average PSNR of 33.5dB, consistently surpassing FisheyeGS (32.8dB) across all metrics (Tab. J.1). Qualitatively, our reconstructions preserve sharper details in peripheral regions for the indoor scene (Fig. J.1 *Left*) and recover distant structures such as trees in the outdoor scene (Fig. J.1 *Right*). The performance gain (+0.7 dB) lies between the improvements observed in pinhole datasets (e.g., MipNeRF360: +0.3dB; ZipNeRF-cross: +2.0dB) and in highly distorted wide-FoV datasets (e.g., ScanNet++/ZipNeRF 180° diagonal FoV: +0.9–4.8dB), which aligns with Aria’s moderate FoV and distortion level.

K APPENDIX: ADDITIONAL MATERIAL AND RESULTS

K.1 ADDITIONAL MATERIAL.

Code, video, and an HTML page are provided in the Supplementary Material.

K.2 ADDITIONAL RESULTS.



Figure K.1: **Out-of-Distribution (OOD) Region of Fig. 6**. At $\theta \rightarrow 90^\circ$ the pinhole mapping $\tan \theta \rightarrow \infty$ causes 3DGS to collapse to black; 3DGEER instead renders robustly out to 180° , reconstructing the green chair where splatting-based methods such as FisheyeGS fail. Consistently, in the Fig. 6 zoom-in, our method also preserves structures such as the corridor and fridge, the table and door, and the window and cabinet.



Figure K.2: **Extreme-FoV Generalization.** (Left) 3DGEER vs. 3DGS on extreme-FoV pinhole (170°) views, both trained on undistorted central regions. (Right) Zoom-out of Fig. 6 over the complete circular 180° FoV, where 3DGEER generalizes robustly into the out-of-distribution periphery.

Table K.1: **Per-Sequence MipNeRF360 Results for 3DGEER.** Aggregate results are given in Tab. 4.

Method	Metric	Bicycle	Bonsai	Counter	Garden	Kitchen	Stump	Flowers	Room	Treehill
Ours	PSNR $\uparrow$	25.63	32.23	29.05	27.60	31.78	26.83	21.83	32.09	22.83
	SSIM $\uparrow$	0.774	0.946	0.910	0.860	0.931	0.774	0.613	0.946	0.637
	LPIPS $\downarrow$	0.223	0.175	0.195	0.136	0.120	0.246	0.340	0.108	0.344

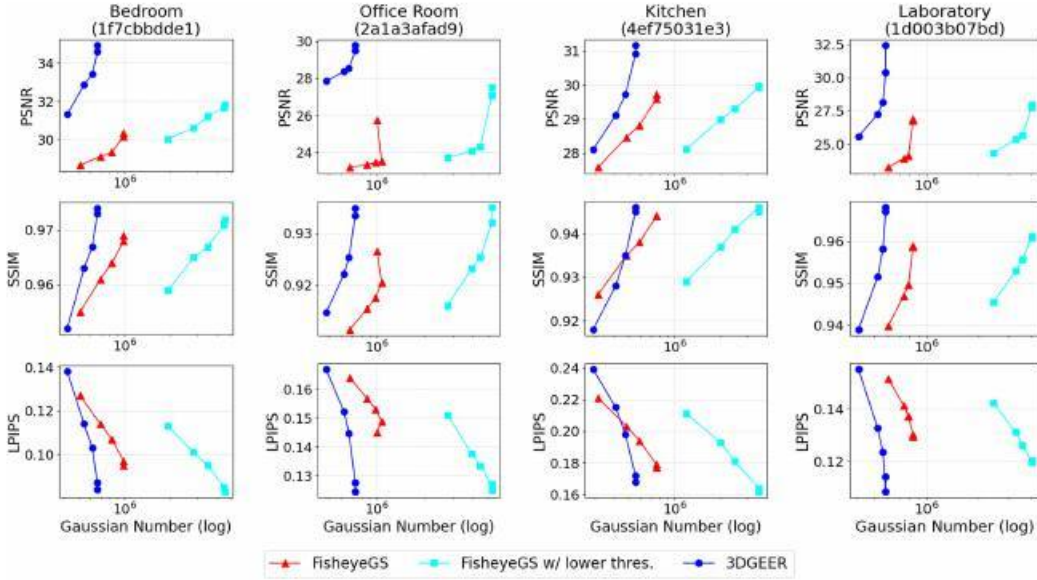


Figure K.3: **PSNR $\uparrow$, SSIM $\uparrow$, and LPIPS $\downarrow$ vs. Gaussian Count.** Across all three metrics, brute-force scaling of the splatting baseline fails to close the projective-exactness gap to 3DGEER.

Table K.2: **Per-Sequence ScanNet++ Results (Full FoV).** Aggregate results are given in Tab. 2. 3DGEER attains the best or tied-best PSNR, SSIM, and LPIPS on every scene.

Scene	EVER			FisheyeGS			3DGUT			3DGEER (Ours)		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.24	0.916	0.164	24.74	0.932	0.146	29.26	0.934	0.147	29.18	0.940	0.141
Office Room	27.80	0.899	0.169	25.75	0.927	0.145	29.37	0.928	0.145	29.78	0.935	0.125
Bedroom	32.60	0.948	0.131	31.95	0.970	0.095	34.65	0.967	0.112	34.94	0.974	0.084
Kitchen	29.77	0.924	0.213	29.72	0.944	0.177	30.78	0.941	0.196	31.17	0.946	0.168
Laboratory	28.96	0.936	0.156	26.91	0.959	0.129	29.12	0.948	0.151	32.43	0.968	0.114



Figure K.4: **Qualitative Results on the ScanNet++ Dataset.** The colored zoom-in boxes highlight the largest differences, concentrated in structural detail and artifact-prone regions. 3DGEER reconstructs the periphery better than FisheyeGS, shows fewer artifacts and sharper detail than EVER, and renders crisper object boundaries and textures—wires, keyboards, wooden floors—than 3DGUT.

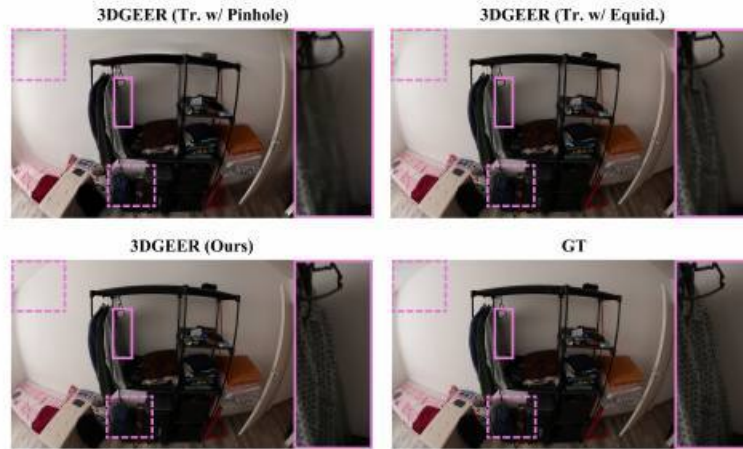


Figure K.5: **Impact of BEAP Supervision.** On ScanNet++, BEAP’s balanced angular sampling yields smoother gradients and better preservation of geometry and texture than pinhole or equidistant sampling (Tab. K.8 reports the full quantitative statistics).

Table K.3: **Scaling-Up Gaussian Counts on ScanNet++**. PSNR $\uparrow$, SSIM $\uparrow$, LPIPS $\downarrow$, and Gaussian count as a function of training iteration, for Bedroom, Office Room, Kitchen, and Laboratory in order. Even with $5\text{--}7\times$ more Gaussians, the scaled FisheyeGS variant trails 3DGEER on PSNR.

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	28.67	0.955	0.127	512K	30.03	0.959	0.113	1914K	31.33	0.952	0.138	422K
7k	29.12	0.961	0.114	695K	30.61	0.965	0.101	2829K	32.87	0.963	0.114	543K
10k	29.36	0.964	0.107	827K	31.21	0.967	0.095	3489K	33.42	0.967	0.103	616K
20k	30.17	0.968	0.097	985K	31.68	0.971	0.085	4467K	34.59	0.973	0.087	664K
30k	30.36	0.969	0.095	985K	31.84	0.972	0.083	4535K	34.94	0.974	0.084	664K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.19	0.911	0.164	677K	23.72	0.916	0.151	2778K	27.84	0.915	0.167	480K
7k	23.33	0.916	0.157	867K	24.06	0.923	0.138	3936K	28.37	0.922	0.152	621K
10k	23.45	0.918	0.153	983K	24.31	0.925	0.133	4481K	28.54	0.925	0.145	668K
20k	23.51	0.921	0.149	1072K	27.07	0.932	0.127	5272K	29.49	0.933	0.128	732K
30k	25.75	0.927	0.145	1001K	27.50	0.935	0.125	5272K	29.78	0.935	0.125	732K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	27.58	0.926	0.221	314K	28.12	0.929	0.211	1191K	28.11	0.918	0.239	292K
7k	28.47	0.935	0.203	485K	28.99	0.937	0.193	1997K	29.11	0.928	0.215	410K
10k	28.82	0.938	0.194	588K	29.29	0.941	0.181	2491K	29.73	0.935	0.198	476K
20k	29.60	0.944	0.179	760K	29.93	0.946	0.164	3563K	30.91	0.945	0.172	552K
30k	29.72	0.944	0.177	760K	29.97	0.945	0.162	3563K	31.17	0.946	0.168	552K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.27	0.940	0.152	602K	24.32	0.945	0.142	2429K	25.59	0.939	0.155	406K
7k	23.94	0.947	0.141	740K	25.36	0.953	0.131	3258K	27.27	0.952	0.133	524K
10k	24.12	0.950	0.137	788K	25.65	0.956	0.126	3569K	28.14	0.958	0.124	562K
20k	26.76	0.959	0.130	838K	27.77	0.961	0.120	4035K	30.39	0.967	0.108	582K
30k	26.91	0.959	0.129	838K	27.97	0.961	0.120	4035K	32.43	0.968	0.114	582K

Table K.4: **Runtime Analysis vs. Splatting Baselines**. Stage-wise and total per-frame timings comparing 3DGEER with efficient splatting-based methods on MipNeRF360 and ScanNet++. The reduced duplication and sorting cost from PBF association offsets the slightly higher rendering cost, yielding competitive totals.

Dataset	Method	Avg. Timings (ms) $\downarrow$				
		Prep.	Dup.	Sort	Render	Total
MipNeRF360	3DGS	0.32	0.27	0.65	1.40	2.92
	3DGEER (Ours)	0.37	0.17	0.27	2.10	3.06
ScanNet++	FisheyeGS	0.10	0.63	1.45	2.33	4.70
	3DGEER (Ours)	0.13	0.26	0.59	2.89	3.98

Table K.5: **Association-Method Runtime on ScanNet++**. PBF association is compared against the splatting-based EWA and UT schemes—each with the SnugBox accelerator—all integrated into the same 3DGEER pipeline. PBF attains the lowest per-stage and total timings and the highest FPS.

3DGEER Asso.	Avg. Timings (ms) ↓					Avg. FPS ↑
	Prep.	Dup.	Sort	Render	Total	
EWA	0.13	1.17	3.27	12.69	17.84	56
+ SnugBox	0.13	0.60	1.42	6.22	8.68	115
UT	0.13	0.71	1.94	7.97	11.16	90
+ SnugBox	0.13	0.28	0.64	3.16	4.38	228
PBF (Ours)	0.13	0.26	0.59	2.89	3.98	251

Table K.6: **Impact of PBF-CSF Association**. Evaluation uses the artifact-sensitive LPIPS metric. Cross-model robustness is probed by rendering each association scheme on models trained under the other schemes. We additionally report rendering memory, the average number of Gaussians associated per tile, and the in-method vs. cross-method gap Δ LPIPS as a measure of consistency and generalization. PBF is best on every column, with the smallest Δ LPIPS by a wide margin.

Asso. Method	Gaus. ↓ per Tile		Mem. ↓	LPIPS ↓	Cross-LPIPS ↓	Δ LPIPS ↓
	mean	std	(GB)			(1e-2)
EWA	2203	1447	2.2	0.1250	0.1321	0.71
UT	1377	1058	1.4	0.1249	0.1302	0.53
PBF (Ours)	475	427	0.63	0.1245	0.1251	0.06

Table K.7: **Per-Sequence Training-Time Transmittance Replacement**. Aggregate results are given in Tab. 5 the projective-exact formulation attains the best quality with the fewest Gaussians on every scene.

Scene	3DGS Splats + PBF				FisheyeGS Splats + PBF				3DGEER (Ours)			
	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)
Storage	19.27	0.859	0.173	1293	24.75	0.933	0.150	781	29.18	0.940	0.141	429
Office Room	20.89	0.848	0.182	1527	25.89	0.928	0.150	1119	29.78	0.935	0.125	732
Bedroom	27.39	0.923	0.119	1542	31.96	0.970	0.096	1079	34.94	0.974	0.084	664
Kitchen	25.15	0.890	0.239	1241	29.87	0.949	0.182	801	31.17	0.946	0.168	552
Laboratory	21.60	0.869	0.173	1379	27.03	0.962	0.131	823	32.43	0.968	0.114	582

Table K.8: **Per-Sequence Impact of Supervision Space**. Aggregate results are given in Tab. 6. BEAP supervision is best or tied-best on nearly every scene and metric.

Supervision Space Scene	3DGEER w/o BEAP Central Perspective			3DGEER w/o BEAP Perspective			3DGEER w/o BEAP Equidistant			3DGEER (Ours) BEAP		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.42	0.931	0.143	19.74	0.834	0.302	29.03	0.938	0.142	29.18	0.940	0.141
Office Room	28.90	0.927	0.133	21.12	0.825	0.291	29.71	0.932	0.126	29.78	0.935	0.125
Bedroom	33.86	0.965	0.090	24.38	0.882	0.240	34.60	0.973	0.096	34.94	0.974	0.084
Kitchen	30.65	0.941	0.172	23.23	0.889	0.338	31.23	0.941	0.177	31.17	0.946	0.168
Laboratory	27.35	0.951	0.118	17.08	0.834	0.329	30.65	0.956	0.135	32.43	0.968	0.114

Table K.9: **Per-Sequence ZipNeRF Results.** Alameda, Berlin, London, and NYC in order. Aggregate results are given in Tab. 3

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	20.74	23.64	0.820	0.802	0.223	0.277	17.39	23.74	0.736	0.839	0.265	0.240
EVER	22.46	23.06	0.852	0.777	0.144	0.274	20.06	22.83	0.789	0.794	0.213	0.250
3DGUT	21.78	22.39	0.841	0.718	0.193	0.368	17.03	22.58	0.588	0.765	0.329	0.311
Ours	24.60	25.00	0.857	0.839	0.154	0.241	21.49	25.21	0.792	0.807	0.232	0.277

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.25	28.25	0.895	0.924	0.206	0.209	20.08	27.86	0.848	0.929	0.233	0.194
EVER	25.26	26.97	0.912	0.907	0.155	0.221	23.18	27.72	0.877	0.913	0.196	0.212
3DGUT	25.37	27.02	0.904	0.876	0.197	0.288	18.57	27.43	0.730	0.898	0.297	0.249
Ours	25.51	29.15	0.916	0.925	0.166	0.194	23.58	29.51	0.878	0.915	0.219	0.228

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	23.57	26.65	0.829	0.861	0.253	0.254	20.83	27.33	0.768	0.883	0.288	0.229
EVER	24.59	26.83	0.823	0.859	0.218	0.223	23.14	26.98	0.793	0.860	0.268	0.235
3DGUT	25.61	26.25	0.854	0.803	0.221	0.336	19.82	26.41	0.669	0.833	0.313	0.291
Ours	26.60	27.84	0.895	0.882	0.142	0.230	23.95	27.15	0.840	0.846	0.217	0.289

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.15	27.21	0.890	0.885	0.162	0.215	19.41	27.50	0.811	0.907	0.203	0.183
EVER	28.39	27.71	0.931	0.860	0.095	0.228	25.22	28.27	0.879	0.883	0.154	0.192
3DGUT	26.31	26.69	0.917	0.817	0.121	0.302	19.00	27.42	0.659	0.866	0.263	0.228
Ours	28.26	28.49	0.922	0.906	0.097	0.190	24.52	28.55	0.874	0.885	0.168	0.220